



(12) 发明专利申请

(10) 申请公布号 CN 116562377 A

(43) 申请公布日 2023. 08. 08

(21) 申请号 202310520179.2

(22) 申请日 2023.05.09

(71) 申请人 工银瑞信基金管理有限公司

地址 100033 北京市西城区金融大街5号新
盛大厦A座

(72) 发明人 李胜浩 邓皓友 杨帆 刘丽丽

(74) 专利代理机构 中科专利商标代理有限责任
公司 11021

专利代理师 张琛

(51) Int.Cl.

G06N 7/01 (2023.01)

G06N 20/00 (2019.01)

G06F 30/20 (2020.01)

G06N 3/084 (2023.01)

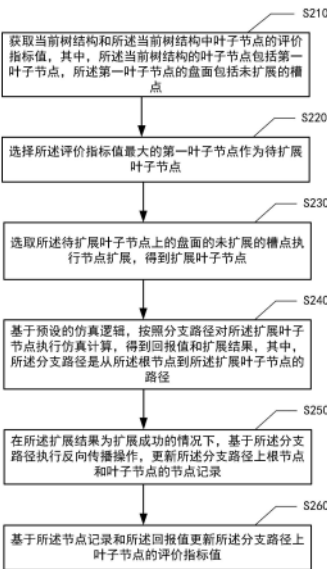
权利要求书5页 说明书25页 附图11页

(54) 发明名称

因子挖掘方法、装置、设备、存储介质和程序
产品

(57) 摘要

本公开提供了一种因子挖掘方法、装置、设备、存储介质和程序产品,可以应用于人工智能技术领域。该因子挖掘方法由蒙特卡洛树实现,蒙特卡洛树包括一个根节点和多个叶子节点,叶子节点包括盘面和节点记录,盘面包括算子和特征构成,盘面包括多个槽点,该方法包括:获取当前树结构和评价指标值;选择评价指标值最大的叶子节点作为待扩展叶子节点;选取待扩展叶子节点上的盘面的未扩展的槽点执行节点扩展,得到扩展叶子节点;按照路径对扩展叶子节点执行仿真计算,得到回报值和扩展结果;在扩展结果为扩展成功的情况下,基于路径执行反向传播操作,更新路径上根节点和叶子节点的节点记录;基于节点记录和回报值更新路径上叶子节点的评价指标值。



1. 一种因子挖掘方法,其特征在于,所述因子挖掘方法由蒙特卡洛树实现,所述蒙特卡洛树包括一个根节点和多个叶子节点,所述叶子节点包括盘面和节点记录,所述盘面包括算子和特征,所述盘面包括多个槽点,所述槽点包括已扩展和未扩展,

所述方法包括:

获取当前树结构和所述当前树结构中叶子节点的评价指标值,其中,所述当前树结构的叶子节点包括第一叶子节点,所述第一叶子节点的盘面包括未扩展的槽点;

选择所述评价指标值最大的第一叶子节点作为待扩展叶子节点;

选取所述待扩展叶子节点上的盘面的未扩展的槽点执行节点扩展,得到扩展叶子节点;

基于预设的仿真逻辑,按照分支路径对所述扩展叶子节点执行仿真计算,得到回报值和扩展结果,其中,所述分支路径是从所述根节点到所述扩展叶子节点的路径;

在所述扩展结果为扩展成功的情况下,基于所述分支路径执行反向传播操作,更新所述分支路径上根节点和叶子节点的节点记录;以及

基于所述节点记录和所述回报值更新所述分支路径上叶子节点的评价指标值。

2. 根据权利要求1所述的方法,其中,所述基于预设的仿真逻辑,按照分支路径对所述扩展叶子节点执行仿真计算,得到回报值和扩展结果,包括:

基于所述分支路径,按照从所述根节点到所述扩展叶子节点的顺序执行因子衍生,得到因子矩阵,其中,所述因子矩阵包括多个因子数值和所述因子数值对应的因子标签;

按照所述多个因子数值的大小,通过排位矩阵算子,对所述因子矩阵中的所述因子标签执行排序,得到排序因子向量;

获取基准向量,所述基准向量是基准标签的排名,所述基准标签与所述因子标签一一对应;以及

基于所述排序因子向量中的因子标签和所述基准向量中的基准标签的位差,计算所述回报值。

3. 根据权利要求2所述的方法,其中,所述根节点包括根节点盘面,

所述基于所述分支路径,按照从所述根节点到所述扩展叶子节点的顺序执行因子衍生,得到因子矩阵,其中,所述因子矩阵包括多个因子数值和所述因子数值对应的因子标签,包括:

对于所述根节点盘面,通过预设的数据特征和聚合算子对收集到的高频时序数据执行降维处理,得到第一降维矩阵;以及

基于预设的一元算子,对所述第一降维矩阵执行计算,得到所述因子矩阵。

4. 根据权利要求3所述的方法,其中,所述叶子节点包括连续算盘面和新特征盘面,

所述基于所述分支路径,按照从所述根节点到所述扩展叶子节点的顺序执行因子衍生,得到因子矩阵,其中,所述因子矩阵包括多个因子数值和所述因子数值对应的因子标签,还包括:

对于所述连续算盘面,通过预设的一元算子,对上一叶子节点生成的因子矩阵执行计算,得到新的因子矩阵;

对于所述连续算盘面,通过预设的数据特征和聚合算子对收集到的高频时序数据执行降维处理,得到第二降维矩阵;以及

基于预设的二元算子,对所述第二降维矩阵和上一叶子节点生成的因子矩阵执行计算,得到新的因子矩阵。

5.根据权利要求2-4中任一项所述的方法,其中,所述基于所述排序因子向量中的因子标签和所述基准向量中的基准标签的位差,计算所述回报值,包括:

获取所述因子矩阵的数据量或者所述基准向量的数据量;以及
通过所述位差和所述数据量,计算所述回报值。

6.根据权利要求1或2所述的方法,其中,所述基于预设的仿真逻辑,按照分支路径对所述扩展叶子节点执行仿真计算,得到回报值和扩展结果,还包括:

判断所述扩展叶子节点是否符合预设的扩展成功条件,所述预设的扩展成功条件包括多个扩展子条件,所述多个扩展子条件用以确定所述扩展叶子节点的可挖掘性;以及

在所述扩展叶子节点符合所述多个扩展子条件中任意一个的情况下,判定所述扩展结果为成功。

7.根据权利要求6所述的方法,其中,所述多个扩展子条件包括的验证方法包括:

判断所述扩展叶子节点的回报值是否处于预设回报值区间;以及

在所述扩展叶子节点的回报值处于所述预设回报值区间内的情况下,确定所述扩展叶子节点符合所述扩展子条件。

8.根据权利要求6所述的方法,其中,所述多个扩展子条件包括的验证方法还包括:

基于所述分支路径,按照从所述扩展叶子节点到所述根节点的顺序,回溯预设的个数的叶子节点的回报值;

基于所述扩展叶子节点的回报值和所述预设的个数的叶子节点的回报值,判断所述回报值是否处于增长趋势;以及

在所述回报值处于增长趋势的情况下,确定所述扩展叶子节点符合所述扩展子条件。

9.根据权利要求6所述的方法,其中,所述多个扩展子条件包括的验证方法还包括:

基于快速走步算子,从所述扩展叶子节点向下随机探索预设步数,得到所述扩展叶子节点下层路径的多个叶子节点的回报值;

基于所述扩展叶子节点的回报值和所述扩展叶子节点下层路径的多个叶子节点,判断所述回报值是否处于增长趋势;以及

在所述回报值处于增长趋势的情况下,确定所述扩展叶子节点符合所述扩展子条件。

10.根据权利要求1所述的方法,其中,所述基于预设的仿真逻辑,按照分支路径对所述扩展叶子节点执行仿真计算,得到回报值和扩展结果,是基于预设的CUDA计算框架执行的,其中,所述预设的CUDA计算框架包括Host端和Device端,

所述基于预设的仿真逻辑,按照分支路径对所述扩展叶子节点执行仿真计算,得到回报值和扩展结果,包括:

由所述CUDA计算框架,基于所述分支路径,按照从所述根节点到所述扩展叶子节点的顺序执行因子衍生,得到因子矩阵,其中,所述因子矩阵包括多个因子数值和所述因子数值对应的因子标签;

由所述Device端,按照所述多个因子数值的大小,通过排位矩阵算子,对所述因子矩阵中的所述因子标签执行排序,得到排序因子向量;

由所述Host端,获取基准向量,所述基准向量是基准标签的排名,所述基准标签与所述

因子标签一一对应;以及

由所述Device端,基于所述排序因子向量中的因子标签和所述基准向量中的基准标签的位差,计算所述回报值。

11. 根据权利要求10所述的方法,其中,所述根节点包括根节点盘面,

所述由所述CUDA计算框架,基于所述分支路径,按照从所述根节点到所述扩展叶子节点的顺序执行因子衍生,得到因子矩阵,其中,所述因子矩阵包括多个因子数值和所述因子数值对应的因子标签,包括:

对于所述根节点盘面,由所述Host端,通过预设的数据特征和聚合算子对收集到的高频时序数据执行降维处理,得到第一降维矩阵;以及

由所述Device端,基于预设的一元算子,对所述第一降维矩阵执行计算,得到所述因子矩阵。

12. 根据权利要求11所述的方法,其中,所述叶子节点包括连续算盘面和新特征盘面,

所述由所述CUDA计算框架,基于所述分支路径,按照从所述根节点到所述扩展叶子节点的顺序执行因子衍生,得到因子矩阵,其中,所述因子矩阵包括多个因子数值和所述因子数值对应的因子标签,还包括:

对于所述连续算盘面,由所述Device端,通过预设的一元算子,对上一叶子节点生成的因子矩阵执行计算,得到新的因子矩阵;

对于所述连续算盘面,由所述Host端,通过预设的数据特征和聚合算子对收集到的高频时序数据执行降维处理,得到第二降维矩阵;以及

由所述Device端,基于预设的二元算子,对所述第二降维矩阵和上一叶子节点生成的因子矩阵执行计算,得到新的因子矩阵。

13. 根据权利要求10-12中任一项所述的方法,其中,所述由所述Device端,基于所述排序因子向量中的因子标签和所述基准向量中的基准标签的位差,计算所述回报值,包括:

由所述Device端,获取所述因子矩阵的数据量或者所述基准向量的数据量;以及

由所述Device端,通过所述位差和所述数据量,计算所述回报值。

14. 根据权利要求10所述的方法,其中,所述基于预设的仿真逻辑,按照分支路径对所述扩展叶子节点执行仿真计算,得到回报值和扩展结果,还包括:

由所述CUDA计算框架,判断所述扩展叶子节点是否符合预设的扩展成功条件,所述预设的扩展成功条件包括多个扩展子条件,所述多个扩展子条件用以确定所述扩展叶子节点的可挖掘性;以及

由所述CUDA计算框架,在所述扩展叶子节点符合所述多个扩展子条件中任意一个的情况下,判定所述扩展结果为成功。

15. 根据权利要求14所述的方法,其中,所述多个扩展子条件包括的验证方法包括:

由所述Host端,判断所述扩展叶子节点的回报值是否处于预设回报值区间;以及

由所述Host端,在所述扩展叶子节点的回报值处于所述预设回报值区间内的情况下,确定所述扩展叶子节点符合所述扩展子条件。

16. 根据权利要求14所述的方法,其中,所述多个扩展子条件包括的验证方法还包括:

由所述Host端,基于所述分支路径,按照从所述扩展叶子节点到所述根节点的顺序,回溯预设的个数的叶子节点的回报值;

由所述Host端,基于所述扩展叶子节点的回报值和所述预设的个数的叶子节点的回报值,判断所述回报值是否处于增长趋势;以及

由所述Host端,在所述回报值处于增长趋势的情况下,确定所述扩展叶子节点符合所述扩展子条件。

17. 根据权利要求14所述的方法,其中,所述多个扩展子条件包括的验证方法还包括:

由所述Device端,基于快速走步算子,从所述扩展叶子节点向下随机探索预设步数,得到所述扩展叶子节点下层路径的多个叶子节点的回报值;

由所述Host端,基于所述扩展叶子节点的回报值和所述扩展叶子节点下层路径的多个叶子节点,判断所述回报值是否处于增长趋势;以及

由所述Host端,在所述回报值处于增长趋势的情况下,确定所述扩展叶子节点符合所述扩展子条件。

18. 根据权利要求1、2、3、4、7、8、9、10、11、12、14、15、16以及17中任一项所述的方法,其中,所述节点记录包括:效果提升次数、节点被访问次数以及计算总数,

所述在所述扩展结果为扩展成功的情况下,基于所述分支路径执行反向传播操作,更新所述分支路径上根节点和叶子节点的节点记录,包括:

更新所述分支路径上根节点和叶子节点的所述效果提升次数、所述节点被访问次数以及所述计算总数。

19. 根据权利要求18所述的方法,其中,所述回报值用以评价所述叶子节点的深度,

所述基于所述节点记录和所述回报值更新所述分支路径上叶子节点的评价指标值,包括:

基于所述效果提升次数和所述节点被访问次数,计算所述叶子节点的开发能力值,所述开发能力值用以评价所述叶子节点的深度;

基于所述节点被访问次数和所述计算总数,计算所述叶子节点的勘探能力值,所述勘探能力值用以评价所述叶子节点的广度;以及

基于所述回报值、所述开发能力值以及所述勘探能力值,通过加权的方式执行计算,得到所述评价指标值。

20. 根据权利要求1、2、3、4、7、8、9、10、11、12、14、15、16、17以及19中任一项所述的方法,其中,所述选择所述评价指标值最大的第一叶子节点作为待扩展叶子节点,包括:

在多个所述评价指标值相同的情况下,随机挑选第一叶子节点作为待扩展叶子节点。

21. 根据权利要求4或12所述的方法,其中,所述选取所述待扩展叶子节点上的盘面的未扩展的槽点执行节点扩展,得到扩展叶子节点,包括:

通过加权抽取,随机对所述新特征盘面和所述连续算盘面中未扩展的槽点执行抽取;以及

基于抽取到的未扩展的槽点执行节点扩展,得到扩展叶子节点。

22. 根据权利要求1、2、3、4、7、8、9、10、11、12、14、15、16、17以及19中任一项所述的方法,其中,所述叶子节点还包括节点深度,

在所述基于所述节点记录和所述回报值更新所述分支路径上叶子节点的评价指标值后,还包括:

判断所述扩展叶子节点的节点深度是否超过预设的深度阈值;以及

在所述扩展叶子节点的节点深度满足预设的深度阈值的情况下,结束因子挖掘。

23.一种因子挖掘装置,其特征在于,所述因子挖掘装置由蒙特卡洛树实现,所述蒙特卡洛树包括一个根节点和多个叶子节点,所述叶子节点包括盘面和节点记录,所述盘面包括算子和特征,所述盘面包括多个槽点,所述槽点包括已扩展和未扩展,

所述装置包括:

数据获取模块,用于获取当前树结构和所述当前树结构中叶子节点的评价指标值,其中,所述当前树结构的叶子节点包括第一叶子节点,所述第一叶子节点的盘面包括未扩展的槽点;

节点选择模块,用于选择所述评价指标值最大的第一叶子节点作为待扩展叶子节点;

节点扩展模块,用于选取所述待扩展叶子节点上的盘面的未扩展的槽点执行节点扩展,得到扩展叶子节点;

仿真计算模块,用于基于预设的仿真逻辑,按照分支路径对所述扩展叶子节点执行仿真计算,得到回报值和扩展结果,其中,所述分支路径是从所述根节点到所述扩展叶子节点的路径;

反向传播模块,用于在所述扩展结果为扩展成功的情况下,基于所述分支路径执行反向传播操作,更新所述分支路径上根节点和叶子节点的节点记录;以及

评价指标更新模块,用于基于所述节点记录和所述回报值更新所述分支路径上叶子节点的评价指标值。

24.一种电子设备,包括:

一个或多个处理器;

存储装置,用于存储一个或多个程序,

其中,当所述一个或多个程序被所述一个或多个处理器执行时,使得所述一个或多个处理器执行根据权利要求1~22中任一项所述的方法。

25.一种计算机可读存储介质,其上存储有可执行指令,该指令被处理器执行时使处理器执行根据权利要求1~22中任一项所述的方法。

26.一种计算机程序产品,包括计算机程序,所述计算机程序被处理器执行时实现根据权利要求1~22中任一项所述的方法。

因子挖掘方法、装置、设备、存储介质和程序产品

技术领域

[0001] 本公开涉及人工智能领域,具体地涉及一种因子挖掘方法、装置、设备、介质和程序产品。

背景技术

[0002] 因子主要由数据和有限个数的算子合成,随着机器学习和数据挖掘技术的不断发展,通过算力合成一些特殊的新因子成为一种趋势。虽然合成技术因子主要为数据和有限算子,但从排列组合的层面来讲极其复杂,故而暴力遍历并不能完成对因子空间搜索的任务,特别在数据量巨大的场景下。而采用遗传算法实现因子挖掘的方法,则在收敛性上存在一些问题。

发明内容

[0003] 鉴于上述问题,本公开提供了针对规模庞大的因子空间中能够提高因子挖掘能力的因子挖掘方法、装置、设备、介质和程序产品。

[0004] 根据本公开的第一个方面,提供了一种因子挖掘方法,包括:所述因子挖掘方法由蒙特卡洛树实现,所述蒙特卡洛树包括一个根节点和多个叶子节点,所述叶子节点包括盘面和节点记录,所述盘面包括算子和特征,所述盘面包括多个槽点,所述槽点包括已扩展和未扩展,所述方法包括:获取当前树结构和所述当前树结构中叶子节点的评价指标值,其中,所述当前树结构的叶子节点包括第一叶子节点,所述第一叶子节点的盘面包括未扩展的槽点;选择所述评价指标值最大的第一叶子节点作为待扩展叶子节点;选取所述待扩展叶子节点上的盘面的未扩展的槽点执行节点扩展,得到扩展叶子节点;基于预设的仿真逻辑,按照分支路径对所述扩展叶子节点执行仿真计算,得到回报值和扩展结果,其中,所述分支路径是从所述根节点到所述扩展叶子节点的路径;在所述扩展结果为扩展成功的情况下,基于所述分支路径执行反向传播操作,更新所述分支路径上根节点和叶子节点的节点记录;以及基于所述节点记录和所述回报值更新所述分支路径上叶子节点的评价指标值。

[0005] 根据本公开的实施例,其中,所述基于预设的仿真逻辑,按照分支路径对所述扩展叶子节点执行仿真计算,得到回报值和扩展结果,包括:基于所述分支路径,按照从所述根节点到所述扩展叶子节点的顺序执行因子衍生,得到因子矩阵,其中,所述因子矩阵包括多个因子数值和所述因子数值对应的因子标签;按照所述多个因子数值的大小,通过排位矩阵算子,对所述因子矩阵中的所述因子标签执行排序,得到排序因子向量;获取基准向量,所述基准向量是基准标签的排名,所述基准标签与所述因子标签一一对应;以及基于所述排序因子向量中的因子标签和所述基准向量中的基准标签的位差,计算所述回报值。

[0006] 根据本公开的实施例,其中,所述根节点包括根节点盘面,所述基于所述分支路径,按照从所述根节点到所述扩展叶子节点的顺序执行因子衍生,得到因子矩阵,其中,所述因子矩阵包括多个因子数值和所述因子数值对应的因子标签,包括:对于所述根节点盘面,通过预设的数据特征和聚合算子对收集到的高频时序数据执行降维处理,得到第一降

维矩阵;以及基于预设的一元算子,对所述第一降维矩阵执行计算,得到所述因子矩阵。

[0007] 根据本公开的实施例,其中,所述叶子节点包括连续算盘面和新特征盘面,所述基于所述分支路径,按照从所述根节点到所述扩展叶子节点的顺序执行因子衍生,得到因子矩阵,其中,所述因子矩阵包括多个因子数值和所述因子数值对应的因子标签,还包括:对于所述连续算盘面,通过预设的一元算子,对上一叶子节点生成的因子矩阵执行计算,得到新的因子矩阵;对于所述连续算盘面,通过预设的数据特征和聚合算子对收集到的高频时序数据执行降维处理,得到第二降维矩阵;以及基于预设的二元算子,对所述第二降维矩阵和上一叶子节点生成的因子矩阵执行计算,得到新的因子矩阵。

[0008] 根据本公开的实施例,其中,所述基于所述排序因子向量中的因子标签和所述基准向量中的基准标签的位差,计算所述回报率,包括:获取所述因子矩阵的数据量或者所述基准向量的数据量;以及通过所述位差和所述数据量,计算所述回报率。

[0009] 根据本公开的实施例,其中,所述基于预设的仿真逻辑,按照分支路径对所述扩展叶子节点执行仿真计算,得到回报值和扩展结果,还包括:判断所述扩展叶子节点是否符合预设的扩展成功条件,所述预设的扩展成功条件包括多个扩展子条件,所述多个扩展子条件用以确定所述扩展叶子节点的可挖掘性;以及在所述扩展叶子节点符合所述多个扩展子条件中任意一个的情况下,判定所述扩展结果为成功。

[0010] 根据本公开的实施例,其中,所述多个扩展子条件包括的验证方法包括:判断所述扩展叶子节点的回报率是否处于预设回报率区间;以及在所述扩展叶子节点的回报率处于所述预设回报率区间内的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0011] 根据本公开的实施例,其中,所述多个扩展子条件包括的验证方法还包括:基于所述分支路径,按照从所述扩展叶子节点到所述根节点的顺序,回溯预设的个数的叶子节点的回报率;基于所述扩展叶子节点的回报值和所述预设的个数的叶子节点的回报率,判断所述回报率是否处于增长趋势;以及在所述回报率处于增长趋势的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0012] 根据本公开的实施例,其中,所述多个扩展子条件包括的验证方法还包括:基于快速走步算子,从所述扩展叶子节点向下随机探索预设步数,得到所述扩展叶子节点下层路径的多个叶子节点的回报率;基于所述扩展叶子节点的回报值和所述扩展叶子节点下层路径的多个叶子节点,判断所述回报率是否处于增长趋势;以及在所述回报率处于增长趋势的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0013] 根据本公开的实施例,其中,所述基于预设的仿真逻辑,按照分支路径对所述扩展叶子节点执行仿真计算,得到回报值和扩展结果,是基于预设的CUDA计算框架执行的,其中,所述预设的CUDA计算框架包括Host端和Device端,所述基于预设的仿真逻辑,按照分支路径对所述扩展叶子节点执行仿真计算,得到回报值和扩展结果,包括:由所述CUDA计算框架,基于所述分支路径,按照从所述根节点到所述扩展叶子节点的顺序执行因子衍生,得到因子矩阵,其中,所述因子矩阵包括多个因子数值和所述因子数值对应的因子标签;由所述Device端,按照所述多个因子数值的大小,通过排位矩阵算子,对所述因子矩阵中的所述因子标签执行排序,得到排序因子向量;由所述Host端,获取基准向量,所述基准向量是基准标签的排名,所述基准标签与所述因子标签一一对应;以及由所述Device端,基于所述排序因子向量中的因子标签和所述基准向量中的基准标签的位差,计算所述回报率。

[0014] 根据本公开的实施例,其中,所述根节点包括根节点盘面,所述由所述CUDA计算框架,基于所述分支路径,按照从所述根节点到所述扩展叶子节点的顺序执行因子衍生,得到因子矩阵,其中,所述因子矩阵包括多个因子数值和所述因子数值对应的因子标签,包括:对于所述根节点盘面,由所述Host端,通过预设的数据特征和聚合算子对收集到的高频时序数据执行降维处理,得到第一降维矩阵;以及由所述Device端,基于预设的一元算子,对所述第一降维矩阵执行计算,得到所述因子矩阵。

[0015] 根据本公开的实施例,其中,所述叶子节点包括连续算盘面和新特征盘面,所述由所述CUDA计算框架,基于所述分支路径,按照从所述根节点到所述扩展叶子节点的顺序执行因子衍生,得到因子矩阵,其中,所述因子矩阵包括多个因子数值和所述因子数值对应的因子标签,还包括:对于所述连续算盘面,由所述Device端,通过预设的一元算子,对上一叶子节点生成的因子矩阵执行计算,得到新的因子矩阵;对于所述连续算盘面,由所述Host端,通过预设的数据特征和聚合算子对收集到的高频时序数据执行降维处理,得到第二降维矩阵;以及由所述Device端,基于预设的二元算子,对所述第二降维矩阵和上一叶子节点生成的因子矩阵执行计算,得到新的因子矩阵。

[0016] 根据本公开的实施例,其中,所述由所述Device端,基于所述排序因子向量中的因子标签和所述基准向量中的基准标签的位差,计算所述回报值,包括:由所述Device端,获取所述因子矩阵的数据量或者所述基准向量的数据量;以及由所述Device端,通过所述位差和所述数据量,计算所述回报值。

[0017] 根据本公开的实施例,其中,所述基于预设的仿真逻辑,按照分支路径对所述扩展叶子节点执行仿真计算,得到回报值和扩展结果,还包括:由所述CUDA计算框架,判断所述扩展叶子节点是否符合预设的扩展成功条件,所述预设的扩展成功条件包括多个扩展子条件,所述多个扩展子条件用以确定所述扩展叶子节点的可挖掘性;以及由所述CUDA计算框架,在所述扩展叶子节点符合所述多个扩展子条件中任意一个的情况下,判定所述扩展结果为成功。

[0018] 根据本公开的实施例,其中,所述多个扩展子条件包括的验证方法包括:由所述Host端,判断所述扩展叶子节点的回报值是否处于预设回报值区间;以及由所述Host端,在所述扩展叶子节点的回报值处于所述预设回报值区间内的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0019] 根据本公开的实施例,其中,所述多个扩展子条件包括的验证方法还包括:由所述Host端,基于所述分支路径,按照从所述扩展叶子节点到所述根节点的顺序,回溯预设的个数的叶子节点的回报值;由所述Host端,基于所述扩展叶子节点的回报值和所述预设的个数的叶子节点的回报值,判断所述回报值是否处于增长趋势;以及由所述Host端,在所述回报值处于增长趋势的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0020] 根据本公开的实施例,其中,所述多个扩展子条件包括的验证方法还包括:由所述Device端,基于快速走步算子,从所述扩展叶子节点向下随机探索预设步数,得到所述扩展叶子节点下层路径的多个叶子节点的回报值;由所述Host端,基于所述扩展叶子节点的回报值和所述扩展叶子节点下层路径的多个叶子节点,判断所述回报值是否处于增长趋势;以及由所述Host端,在所述回报值处于增长趋势的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0021] 根据本公开的实施例,其中,所述节点记录包括:效果提升次数、节点被访问次数以及计算总数,所述在所述扩展结果为扩展成功的情况下,基于所述分支路径执行反向传播操作,更新所述分支路径上根节点和叶子节点的节点记录,包括:更新所述分支路径上根节点和叶子节点的所述效果提升次数、所述节点被访问次数以及所述计算总数。

[0022] 根据本公开的实施例,其中,所述回报值用以评价所述叶子节点的深度,所述基于所述节点记录和所述回报值更新所述分支路径上叶子节点的评价指标值,包括:基于所述效果提升次数和所述节点被访问次数,计算所述叶子节点的开发能力值,所述开发能力值用以评价所述叶子节点的深度;基于所述节点被访问次数和所述计算总数,计算所述叶子节点的勘探能力值,所述勘探能力值用以评价所述叶子节点的广度;以及基于所述回报值、所述开发能力值以及所述勘探能力值,通过加权的方式执行计算,得到所述评价指标值。

[0023] 根据本公开的实施例,其中,所述选择所述评价指标值最大的第一叶子节点作为待扩展叶子节点,包括:在多个所述评价指标值相同的情况下,随机挑选第一叶子节点作为待扩展叶子节点。

[0024] 根据本公开的实施例,其中,所述选取所述待扩展叶子节点上的盘面的未扩展的槽点执行节点扩展,得到扩展叶子节点,包括:通过加权抽取,随机对所述新特征盘面和所述连续算盘面中未扩展的槽点执行抽取;以及基于抽取到的未扩展的槽点执行节点扩展,得到扩展叶子节点。

[0025] 根据本公开的实施例,其中,所述叶子节点还包括节点深度,在所述基于所述节点记录和所述回报值更新所述分支路径上叶子节点的评价指标值后,还包括:判断所述扩展叶子节点的节点深度是否超过预设的深度阈值;以及在所述扩展叶子节点的节点深度满足预设的深度阈值的情况下,结束因子挖掘。

[0026] 本公开的第二个方面,提供了一种因子挖掘装置,包括:所述因子挖掘装置由蒙特卡洛树实现,所述蒙特卡洛树包括一个根节点和多个叶子节点,所述叶子节点包括盘面和节点记录,所述盘面包括算子和特征,所述盘面包括多个槽点,所述槽点包括已扩展和未扩展,所述装置包括:数据获取模块,用于获取当前树结构和所述当前树结构中叶子节点的评价指标值,其中,所述当前树结构的叶子节点包括第一叶子节点,所述第一叶子节点的盘面包括未扩展的槽点;节点选择模块,用于选择所述评价指标值最大的第一叶子节点作为待扩展叶子节点;节点扩展模块,用于选取所述待扩展叶子节点上的盘面的未扩展的槽点执行节点扩展,得到扩展叶子节点;仿真计算模块,用于基于预设的仿真逻辑,按照分支路径对所述扩展叶子节点执行仿真计算,得到回报值和扩展结果,其中,所述分支路径是从所述根节点到所述扩展叶子节点的路径;反向传播模块,用于在所述扩展结果为扩展成功的情况下,基于所述分支路径执行反向传播操作,更新所述分支路径上根节点和叶子节点的节点记录;以及评价指标更新模块,用于基于所述节点记录和所述回报值更新所述分支路径上叶子节点的评价指标值。

[0027] 根据本公开的实施例,其中,所述仿真计算模块,还用于基于所述分支路径,按照从所述根节点到所述扩展叶子节点的顺序执行因子衍生,得到因子矩阵,其中,所述因子矩阵包括多个因子数值和所述因子数值对应的因子标签;按照所述多个因子数值的大小,通过排位矩阵算子,对所述因子矩阵中的所述因子标签执行排序,得到排序因子向量;获取基准向量,所述基准向量是基准标签的排名,所述基准标签与所述因子标签一一对应;以及基

于所述排序因子向量中的因子标签和所述基准向量中的基准标签的位差,计算所述回报值。

[0028] 根据本公开的实施例,其中,所述根节点包括根节点盘面,所述仿真计算模块,还用于对于所述根节点盘面,通过预设的数据特征和聚合算子对收集到的高频时序数据执行降维处理,得到第一降维矩阵;以及基于预设的一元算子,对所述第一降维矩阵执行计算,得到所述因子矩阵。

[0029] 根据本公开的实施例,其中,所述仿真计算模块,还用于对于所述连续算盘面,通过预设的一元算子,对上一叶子节点生成的因子矩阵执行计算,得到新的因子矩阵;对于所述连续算盘面,通过预设的数据特征和聚合算子对收集到的高频时序数据执行降维处理,得到第二降维矩阵;以及基于预设的二元算子,对所述第二降维矩阵和上一叶子节点生成的因子矩阵执行计算,得到新的因子矩阵。

[0030] 根据本公开的实施例,其中,所述仿真计算模块,还用于获取所述因子矩阵的数据量或者所述基准向量的数据量;以及通过所述位差和所述数据量,计算所述回报值。

[0031] 根据本公开的实施例,其中,所述仿真计算模块,还用于判断所述扩展叶子节点是否符合预设的扩展成功条件,所述预设的扩展成功条件包括多个扩展子条件,所述多个扩展子条件用以确定所述扩展叶子节点的可挖掘性;以及在所述扩展叶子节点符合所述多个扩展子条件中任意一个的情况下,判定所述扩展结果为成功。

[0032] 根据本公开的实施例,其中,所述仿真计算模块,还用于判断所述扩展叶子节点的回报值是否处于预设回报值区间;以及在所述扩展叶子节点的回报值处于所述预设回报值区间内的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0033] 根据本公开的实施例,其中,所述仿真计算模块,还用于基于所述分支路径,按照从所述扩展叶子节点到所述根节点的顺序,回溯预设的个数的叶子节点的回报值;基于所述扩展叶子节点的回报值和所述预设的个数的叶子节点的回报值,判断所述回报值是否处于增长趋势;以及在所述回报值处于增长趋势的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0034] 根据本公开的实施例,其中,所述仿真计算模块,还用于基于快速走步算子,从所述扩展叶子节点向下随机探索预设步数,得到所述扩展叶子节点下层路径的多个叶子节点的回报值;基于所述扩展叶子节点的回报值和所述扩展叶子节点下层路径的多个叶子节点,判断所述回报值是否处于增长趋势;以及在所述回报值处于增长趋势的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0035] 根据本公开的实施例,其中,所述仿真计算模块,是基于预设的CUDA计算框架执行的,其中,所述预设的CUDA计算框架包括Host端和Device端,所述仿真计算模块,还用于由所述CUDA计算框架,基于所述分支路径,按照从所述根节点到所述扩展叶子节点的顺序执行因子衍生,得到因子矩阵,其中,所述因子矩阵包括多个因子数值和所述因子数值对应的因子标签;由所述Device端,按照所述多个因子数值的大小,通过排位矩阵算子,对所述因子矩阵中的所述因子标签执行排序,得到排序因子向量;由所述Host端,获取基准向量,所述基准向量是基准标签的排名,所述基准标签与所述因子标签一一对应;以及由所述Device端,基于所述排序因子向量中的因子标签和所述基准向量中的基准标签的位差,计算所述回报值。

[0036] 根据本公开的实施例,其中,所述根节点包括根节点盘面,所述仿真计算模块,还用于对于所述根节点盘面,由所述Host端,通过预设的数据特征和聚合算子对收集到的高频时序数据执行降维处理,得到第一降维矩阵;以及由所述Device端,基于预设的一元算子,对所述第一降维矩阵执行计算,得到所述因子矩阵。

[0037] 根据本公开的实施例,其中,所述仿真计算模块,还用于对于所述连续算盘面,由所述Device端,通过预设的一元算子,对上一叶子节点生成的因子矩阵执行计算,得到新的因子矩阵;对于所述连续算盘面,由所述Host端,通过预设的数据特征和聚合算子对收集到的高频时序数据执行降维处理,得到第二降维矩阵;以及由所述Device端,基于预设的二元算子,对所述第二降维矩阵和上一叶子节点生成的因子矩阵执行计算,得到新的因子矩阵。

[0038] 根据本公开的实施例,其中,所述仿真计算模块,还用于由所述Device端,获取所述因子矩阵的数据量或者所述基准向量的数据量;以及由所述Device端,通过所述位差和所述数据量,计算所述回报值。

[0039] 根据本公开的实施例,其中,所述仿真计算模块,还用于由所述CUDA计算框架,判断所述扩展叶子节点是否符合预设的扩展成功条件,所述预设的扩展成功条件包括多个扩展子条件,所述多个扩展子条件用以确定所述扩展叶子节点的可挖掘性;以及由所述CUDA计算框架,在所述扩展叶子节点符合所述多个扩展子条件中任意一个的情况下,判定所述扩展结果为成功。

[0040] 根据本公开的实施例,其中,所述仿真计算模块,还用于由所述Host端,判断所述扩展叶子节点的回报值是否处于预设回报值区间;以及由所述Host端,在所述扩展叶子节点的回报值处于所述预设回报值区间内的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0041] 根据本公开的实施例,其中,所述仿真计算模块,还用于由所述Host端,基于所述分支路径,按照从所述扩展叶子节点到所述根节点的顺序,回溯预设的个数的叶子节点的回报值;由所述Host端,基于所述扩展叶子节点的回报值和所述预设的个数的叶子节点的回报值,判断所述回报值是否处于增长趋势;以及由所述Host端,在所述回报值处于增长趋势的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0042] 根据本公开的实施例,其中,所述仿真计算模块,还用于由所述Device端,基于快速走步算子,从所述扩展叶子节点向下随机探索预设步数,得到所述扩展叶子节点下层路径的多个叶子节点的回报值;由所述Host端,基于所述扩展叶子节点的回报值和所述扩展叶子节点下层路径的多个叶子节点,判断所述回报值是否处于增长趋势;以及由所述Host端,在所述回报值处于增长趋势的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0043] 根据本公开的实施例,其中,所述节点记录包括:效果提升次数、节点被访问次数以及计算总数,所述反向传播模块,还用于更新所述分支路径上根节点和叶子节点的所述效果提升次数、所述节点被访问次数以及所述计算总数。

[0044] 根据本公开的实施例,其中,所述评价指标更新模块,还用于基于所述效果提升次数和所述节点被访问次数,计算所述叶子节点的开发能力值,所述开发能力值用以评价所述叶子节点的深度;基于所述节点被访问次数和所述计算总数,计算所述叶子节点的勘探能力值,所述勘探能力值用以评价所述叶子节点的广度;以及基于所述回报值、所述开发能力值以及所述勘探能力值,通过加权的方式执行计算,得到所述评价指标值。

[0045] 根据本公开的实施例,其中,所述节点选择模块,还用于在多个所述评价指标值相同的情况下,随机挑选第一叶子节点作为待扩展叶子节点。

[0046] 根据本公开的实施例,其中,所述节点扩展模块,还用于通过加权抽取,随机对所述新特征盘面和所述连续算盘面中未扩展的槽点执行抽取;以及基于抽取到的未扩展的槽点执行节点扩展,得到扩展叶子节点。

[0047] 根据本公开的实施例,其中,所述叶子节点还包括节点深度,所述装置还包括截止模块,用于判断所述扩展叶子节点的节点深度是否超过预设的深度阈值;以及在所述扩展叶子节点的节点深度满足预设的深度阈值的情况下,结束因子挖掘。

[0048] 本公开的第三个方面,提供了一种电子设备,包括:一个或多个处理器;存储器,用于存储一个或多个程序,其中,当所述一个或多个程序被所述一个或多个处理器执行时,使得一个或多个处理器执行上述因子挖掘方法。

[0049] 本公开的第四个方面,还提供了一种计算机可读存储介质,其上存储有可执行指令,该指令被处理器执行时使处理器执行上述因子挖掘方法。

[0050] 本公开的第五个方面,还提供了一种计算机程序产品,包括计算机程序,该计算机程序被处理器执行时实现上述因子挖掘方法。

[0051] 在本公开的实施例中,基于研究人员发现的因子的合成有其继承性的原则,即,一个比较好的低阶因子,可能衍生出更好的高阶因子,因此,在本公开的实施例中通过蒙特卡洛树实现因子挖掘,贴合了因子在合成时具备继承性的特点,使得在基于蒙特卡洛树在生长探索的过程,可以契合因子挖掘中从低阶因子衍生出高阶因子的过程。并且蒙特卡洛树中每一叶子节点中包含的自定义的盘面设计,这改进了原有的蒙特卡洛树搜索算法,使得该新算法匹配了因子的数据特征,特别在针对规模庞大的因子空间时增强了搜索能力,该改进从算法层面解决因子空间搜索难的问题,进而提高了因子合成的有效性的同时减少了算力开销。

附图说明

[0052] 通过以下参照附图对本公开实施例的描述,本公开的上述内容以及其他目的、特征和优点将更为清楚,在附图中:

[0053] 图1示意性示出了根据本公开实施例的因子挖掘方法的应用场景图;

[0054] 图2示意性示出了根据本公开实施例的因子挖掘方法的流程图;

[0055] 图3示意性示出了根据本公开实施例的仿真计算方法的流程图;

[0056] 图4示意性示出了根据本公开实施例的矩阵处理方法的流程图;

[0057] 图5示意性示出了根据本公开实施例的另一种矩阵处理方法的流程图;

[0058] 图6示意性示出了根据本公开实施例的回报值计算方法的流程图;

[0059] 图7示意性示出了根据本公开实施例的另一种仿真计算方法的流程图;

[0060] 图8示意性示出了根据本公开实施例的扩展子条件的验证方法的流程图;

[0061] 图9示意性示出了根据本公开实施例的另一种扩展子条件的验证方法的流程图;

[0062] 图10示意性示出了根据本公开实施例的扩展子条件的验证方法的流程图;

[0063] 图11示意性示出了根据本公开实施例的评价指标值更新方法的流程图;

[0064] 图12示意性示出了根据本公开实施例的MCTS的树扩展示意图;

[0065] 图13示意性示出了根据本公开实施例的CUDA计算架构下Host端和Device端的调度示意图；

[0066] 图14示意性示出了根据本公开实施例的因子挖掘装置的结构框图；以及

[0067] 图15示意性示出了根据本公开实施例的适于实现因子挖掘方法的电子设备的方框图。

具体实施方式

[0068] 以下，将参照附图来描述本公开的实施例。但是应该理解，这些描述只是示例性的，而并非要限制本公开的范围。在下面的详细描述中，为便于解释，阐述了许多具体的细节以提供对本公开实施例的全面理解。然而，明显地，一个或多个实施例在没有这些具体细节的情况下也可以被实施。此外，在以下说明中，省略了对公知结构和技术的描述，以避免不必要地混淆本公开的概念。

[0069] 在此使用的术语仅仅是为了描述具体实施例，而并非意在限制本公开。在此使用的术语“包括”、“包含”等表明了所述特征、步骤、操作和/或部件的存在，但是并不排除存在或添加一个或多个其他特征、步骤、操作或部件。

[0070] 在此使用的所有术语(包括技术和科学术语)具有本领域技术人员通常所理解的含义，除非另外定义。应注意，这里使用的术语应解释为具有与本说明书的上下文相一致的含义，而不应以理想化或过于刻板的方式来解释。

[0071] 在使用类似于“A、B和C等中至少一个”这样的表述的情况下，一般来说应该按照本领域技术人员通常理解该表述的含义来予以解释(例如，“具有A、B和C中至少一个的系统”应包括但不限于单独具有A、单独具有B、单独具有C、具有A和B、具有A和C、具有B和C、和/或具有A、B、C的系统等)。

[0072] 在对本公开实施例进行详细揭示之前，对本公开的实施例将要用到的关键技术术语进行一一说明：

[0073] 蒙特卡洛：简称为MCTS，蒙特卡洛树搜索是在决策空间中随机抽取样本，随后根据搜索结果构建搜索树来解决行动规划问题的一种方法。

[0074] 统一计算设备架构：(Compute Unified Device Architecture, 简称为CUDA)，一种通用并行计算架构，该架构使GPU能够解决复杂的计算问题。

[0075] 一元算子：仅需一个操作数便能够进行计算的算子，例如，平方、根号等。

[0076] 二元算子：需要两个操作数才能够计算的算子，例如，加法、减法等。

[0077] 在投资领域中，以股票的技术分析法为例，其中存在很多金融工程的因子，例如动量、变异系数以及峰度等，这些因子是由量价数据和有限算子合成的。为了发现新的可以用于股票技术分析的因子，可以采用空间搜索算法实现因子搜索。

[0078] 目前常见的空间搜索算法有适合简单博弈场景的Max-Min算法，也有在AlphaGo中取得极大成功的蒙特卡洛树搜索(MCTS)算法。此类算法结合了广度优先搜索和深度优先搜索，故而在搜索空间很大的情况下，仍能最大程度的找到最优解。当然，由于因子空间搜索和对弈类游戏场景存在明显的不同，MCTS算法需要做出大量的改造方可适配当前场景。

[0079] 此外，因子合成空间搜索在算力上有两个问题：一是因子合成基于高频数据，数据量极大，一年的高频数据大约有：数据量=股票数×量价特征×单日小时数×每小时分钟

数 $\times$ 年交易日数 $\approx 4000 \times 6 \times 4 \times 60 \times 261 = 1,503,360,000$;二是因子有效性的评价需要基于较长时序的验证,以保证因子在较广的时段内具备泛化能力。这种两个问题直接带来的问题就是计算量较大,传统的计算框架要想达到预订的搜索深度,则需要大约一年的时间方可计算完成,没有可行性。

[0080] 为了解决现有技术中存在的技术问题,本公开的实施例提供了一种因子挖掘方法,所述因子挖掘方法由蒙特卡洛树实现,所述蒙特卡洛树包括一个根节点和多个叶子节点,所述叶子节点包括盘面和节点记录,所述盘面包括算子和特征,所述盘面包括多个槽点,所述槽点包括已扩展和未扩展,所述方法包括:获取当前树结构和所述当前树结构中叶子节点的评价指标值,其中,所述当前树结构的叶子节点包括第一叶子节点,所述第一叶子节点的盘面包括未扩展的槽点;选择所述评价指标值最大的第一叶子节点作为待扩展叶子节点;选取所述待扩展叶子节点上的盘面的未扩展的槽点执行节点扩展,得到扩展叶子节点;基于预设的仿真逻辑,按照分支路径对所述扩展叶子节点执行仿真计算,得到回报值和扩展结果,其中,所述分支路径是从所述根节点到所述扩展叶子节点的路径;在所述扩展结果为扩展成功的情况下,基于所述分支路径执行反向传播操作,更新所述分支路径上根节点和叶子节点的节点记录;以及基于所述节点记录和所述回报值更新所述分支路径上叶子节点的评价指标值。

[0081] 在本公开的实施例中,基于研究人员发现的因子的合成有其继承性的原则,即,一个比较好的低阶因子,可能衍生出更好的高阶因子,因此,在本公开的实施例中通过蒙特卡洛树实现因子挖掘,贴合了因子在合成时具备继承性的特点,使得在基于蒙特卡洛树在生长探索的过程,可以契合因子挖掘中从低阶因子衍生出高阶因子的过程。并且蒙特卡洛树中每一叶子节点中包含的自定义的盘面设计,这改进了原有的蒙特卡洛树搜索算法,使得该新算法匹配了因子的数据特征,特别在针对规模庞大的因子空间时增强了搜索能力,该改进从算法层面解决因子空间搜索难的问题,进而提高了因子合成的有效性的同时减少了算力开销。

[0082] 图1示意性示出了根据本公开实施例的因子挖掘方法的应用场景图。

[0083] 如图1所示,根据该实施例的应用场景100可以包括终端设备101、102、103、网络104以及服务器105。网络104用以在终端设备101、102、103和服务器105之间提供通信链路的介质。网络104可以包括各种连接类型,例如有线、无线通信链路或者光纤电缆等等。

[0084] 用户可以使用终端设备101、102、103通过网络104与服务器105交互,以接收或发送消息等。终端设备101、102、103上可以安装有各种通讯客户端应用,例如购物类应用、网页浏览器应用、搜索类应用、即时通信工具、邮箱客户端、社交平台软件等(仅为示例)。

[0085] 终端设备101、102、103可以是具有显示屏并且支持网页浏览的各种电子设备,包括但不限于智能手机、平板电脑、膝上型便携计算机和台式计算机等等。

[0086] 服务器105可以是提供各种服务的服务器,例如对用户利用终端设备101、102、103所浏览的网站提供支持的后台管理服务器(仅为示例)。后台管理服务器可以对接收到的用户请求等数据进行分析等处理,并将处理结果(例如根据用户请求获取或生成的网页、信息、或数据等)反馈给终端设备。

[0087] 需要说明的是,本公开实施例所提供的因子挖掘方法一般可以由服务器105执行。相应地,本公开实施例所提供的因子搜索装置一般可以设置于服务器105中。本公开实施例

所提供的因子挖掘方法也可以由不同于服务器105且能够与终端设备101、102、103和/或服务器105通信的服务器或服务器集群执行。相应地,本公开实施例所提供的因子搜索装置也可以设置于不同于服务器105且能够与终端设备101、102、103和/或服务器105通信的服务器或服务器集群中。

[0088] 应该理解,图1中的终端设备、网络和服务器的数目仅仅是示意性的。根据实现需要,可以具有任意数目的终端设备、网络和服务器。

[0089] 以下将基于图1描述的场景,通过图2~图13对公开实施例的因子挖掘方法进行详细描述。

[0090] 图2示意性示出了根据本公开实施例的因子挖掘方法的流程图。

[0091] 如图2所示,该实施例的因子挖掘方法包括操作S210~操作S260,该因子挖掘方法可以由服务器105执行。

[0092] 根据本公开的实施例,所述因子挖掘方法由蒙特卡洛树实现,所述蒙特卡洛树包括一个根节点和多个叶子节点,所述叶子节点包括盘面和节点记录,所述盘面包括算子和特征,所述盘面包括多个槽点,所述槽点包括已扩展和未扩展。

[0093] 具体的,在金融领域,因子也可称为技术因子,合成的技术因子主要通过数据和算子实现的。而对于蒙特卡洛树搜索算法,该算法是一种通过在决策空间随机取样并根据取样结果构建搜索树的方法,该算法多用于棋盘游戏中。传统的蒙特卡洛树搜索算法一般包括以下过程:1.选择过程,2.扩展过程,3.模拟过程,4.反向传播过程。其中,1.选择过程:是指每次进行蒙特卡洛树搜索时,从根节点开始,根据子节点选择策略递归地选择子节点,直至到达某一个满足条件的节点L,将该节点L作为亟待扩展的节点,该节点L可以是存在子节点的节点,也可以是不存在子节点的节点。2.扩展过程:是指当选择出节点L作为亟待扩展的节点后,随机从行动空间初始化出一个行动集,随后从该行动集中随机选择一个行动进行模拟过程。3.模拟过程:扩展过程结束后,通过一个默认的策略随机走棋直到终端节点T。4.反向传播:根据终端节点T到根节点的路径,更新该路径上所有节点的相关信息。

[0094] 由于因子是通过数据和算子合成的,那么,为了反映因子的合成情况,可以设计盘面(也可称为shape),并由盘面来指导因子合成,因此,盘面必须包括数据和算子,例如,盘面如下表一所示:

[0095]

	数据1	数据2	
算子1	未扩展	已扩展	
算子2	未扩展	未扩展	
.....			

[0096] 表一

[0097] 其中,槽点(可以成为slot)包括扩展和未扩展两项属性,该属性的实际意义在于,反映出某个算子和某个数据是否结合。在蒙特卡洛树中,对于未扩展的槽点,其意义在于当前树节点(包括根节点和叶子节点)不存在由某算子和某数据结合扩展出的子节点;对于已扩展的槽点,其意义在于当前树节点(包括根节点和叶子节点)存在由某算子和某数据结合扩展出的子节点。换句话说,槽点可以看做是一个地址,该地址指向的一个算子和一个数据的合成式,通过属性查看该合成式是否扩展成为节点,例如,通过某个未扩展的槽点,可以确定出该槽点指向合成因子式“算子1和数据1”进行合成。

[0098] 那么,综上所述,盘面可以作用于因子挖掘领域蒙特卡洛树探索的全流程中,即,可以应用于1.选择过程,2.扩展过程,3.模拟过程,4.反向传播过程中。

[0099] 在操作S210中,获取当前树结构和所述当前树结构中叶子节点的评价指标值,其中,所述当前树结构的叶子节点包括第一叶子节点,所述第一叶子节点的盘面包括未扩展的槽点。

[0100] 具体的,当前的树结构可以通过蒙特卡洛树搜索算法探索过多轮的树结构,即当前树结构中包括一个根节点和多个叶子节点(该多个叶子节点可以是不同层次的,并非都是根节点的下一层的子节点)。在本轮因子挖掘的过程中,获取到的当前树结构中的各评价指标值已在上一轮因子挖掘的过程中更新,直接获取记录好的评价指标值即可。需要说明的是,评价指标的作用在于平衡不同的能力和搜索诉求,而对于根节点而言,是不具备评价指标值的,因此,该操作S210讨论的是当前树结构是处于非初始阶段,选择扩展的对象也是叶子节点,因此,当前树结构不是指初始的树结构,即当前树结构中不能仅有一个根节点。

[0101] 在操作S220中,选择所述评价指标值最大的第一叶子节点作为待扩展叶子节点。

[0102] 根据本公开的实施例,其中,所述选择所述评价指标值最大的第一叶子节点作为待扩展叶子节点,包括:在多个所述评价指标值相同的情况下,随机挑选第一叶子节点作为待扩展叶子节点。

[0103] 在执行蒙特卡洛树搜索时,存在充分探索的节点和未充分探索的节点,其中,充分探索的节点是指该节点的盘面中所有槽点为已扩展,换句话说,充分探索的节点不具备再扩展的意义;其中,未充分探索的节点是指该节点的盘面中存在未扩展的槽点。因此,需要选择还未充分探索的节点(即上述第一叶子节点)进行扩展,在众多的第一叶子节点中,选择评价指标值最大的那个第一叶子节点。当然,若出现多个同数值(评价指标值的数值)的叶子节点,且该数值同为最大,则采用随机挑选的方式,从这几个第一叶子节点中随机选择出任意一个第一叶子节点进行进一步地探索扩展。

[0104] 在操作S230中,选取所述待扩展叶子节点上的盘面的未扩展的槽点执行节点扩展,得到扩展叶子节点。

[0105] 具体的,可以通过参考盘面维度,通过槽点(Slot)坐标可以确定该分支的算子、特征的内容。其中,在一个盘面中存在多个属性为未扩展的槽点时,槽点的选取的方式则为随机。

[0106] 根据本公开的实施例,其中,所述根节点包括根节点盘面,所述叶子节点包括连续算盘面和新特征盘面。

[0107] 具体的,盘面是为了应对不同的合成可能性,为此,针对不同的节点,所设计的盘面也不一样。

[0108] 对于根节点而言,根节点的盘面仅包括根节点盘面,该根节点盘面设计为一个三维盘面,根节点盘面=(聚合算子,一元算子,数据特征),即shape=(group,operator,feature)。其中,聚合算子作用于一组数据,并且对一组数据返回一个值,换句话说,聚合算子的作用在于对大批量的数据进行压缩和降维,该聚合算子可以是例如中间值、中位数、最大最小值、均值、方差等;其中,一元算子是指只需要一个操作数的算子,其中,所述一元算子可以例如:指数算子、幂算子、取反算子等,可以理解的是,在根节点时,获取的操作数(矩

阵的形式) 仅是一组数据, 这组数据无法内部相互进行计算, 仅能够通过一元算子进行计算; 其中, 特征是指数据的特征, 在不同的领域中该特征不同, 例如在股票投资领域, 该特征可以是最高价、最低价、收盘价等。上述聚合算子和上述特征可以结合对数据进行降维。

[0109] 例如, 根节点盘面可以如下表二所示:

[0110]

	特征1	特征2	
一元算子1	未扩展	已扩展	
一元算子2	未扩展	未扩展	
.....			

[0111] 表二

[0112] 对于叶子节点而言, 叶子节点的盘面包括连续算盘面和新特征盘面, 其中, 连续算盘面设计为一个一维盘面, 连续算盘面 = (一元算子), 即 $\text{shape} = (\text{operator})$, 表示通过上一树节点 (可以是根节点也可以是叶子节点) 的计算结果后, 再级联一个一元算子以输出性的结果。可以理解的是, 由于连续算盘面无需引入其他新的数据, 仅对上一树节点的数据进一步进行因子合成。

[0113] 例如, 连续算盘面可以如下表三所示:

[0114]

一元算子1	一元算子2	一元算子3	
已扩展	未扩展	已扩展	

[0115] 表三

[0116] 其中, 新特征盘面设计为一个三维盘面, 新特征盘面 = (聚合算子, 二元算子, 新特征), 即 $\text{shape} = (\text{group}, \text{operator}, \text{feature})$, 表示引入新的特征的数据, 再同上一树节点 (可以是根节点也可以是叶子节点) 的计算结果联合计算, 输出新的结果。

[0117] 例如, 新特征盘面可以如下表四所示:

[0118]

	新特征1	新特征2	
二元算子1	未扩展	已扩展	
二元算子2	未扩展	未扩展	
.....			

[0119] 表四

[0120] 通过结合上述根节点盘面、连续算盘面以及新特征盘面, 可以组合成为任意的因子式, 举个例子, 假设现有特征的数据A, 那么根节点盘面通过一元算子“ 2 ”计算后得到“ A^2 ”, 再通过新特征盘面引入新特征的数据B通过二元算子“/”计算后得到“ A^2/B ”, 当然, 还可以是通过连续算盘面通过一元算子“-”对 A^2 进行计算后得到“- A^2 ”。该“ A^2 ”、“ A^2/B ”以及“- A^2 ”都称作是因子式或合成因子式。

[0121] 根据本公开的实施例, 其中, 所述选取所述待扩展叶子节点上的盘面的未扩展的槽点执行节点扩展, 得到扩展叶子节点, 包括: 通过加权抽取, 随机对所述新特征盘面和所述连续算盘面中未扩展的槽点执行抽取; 以及基于抽取到的未扩展的槽点执行节点扩展, 得到扩展叶子节点。

[0122] 加权抽取是一种公平的抽取方式, 例如, 对于一个叶子节点, 具有两个盘面 (即新特征盘面和连续算盘面), 称作盘面A和盘面B, 其中, 盘面A有500个槽点, 盘面B只有50个槽

点,那么在随机抽取时,应该兼顾公平性,通过90%的概率探索盘面A,10%的概率探索盘面B,这就是加权抽取。

[0123] 在操作S240中,基于预设的仿真逻辑,按照分支路径对所述扩展叶子节点执行仿真计算,得到回报值和扩展结果,其中,所述分支路径是从所述根节点到所述扩展叶子节点的路径。

[0124] 具体的,该预设的仿真逻辑则是通过确定好从根节点到新增的扩展叶子节点的分支路径,通过结合实际的特征数据进行计算,从根节点一轮一轮地进行仿真计算,计算至该扩展叶子节点,直至输出该新增的扩展叶子节点的回报值和扩展结果,其中,扩展结果包括成功和失败。

[0125] 在操作S250中,在所述扩展结果为扩展成功的情况下,基于所述分支路径执行反向传播操作,更新所述分支路径上根节点和叶子节点的节点记录。

[0126] 根据本公开的实施例,其中,所述节点记录包括:效果提升次数、节点被访问次数以及计算总数,所述在所述扩展结果为扩展成功的情况下,基于所述分支路径执行反向传播操作,更新所述分支路径上根节点和叶子节点的节点记录,包括:更新所述分支路径上根节点和叶子节点的所述效果提升次数、所述节点被访问次数以及所述计算总数。

[0127] 图12示意性示出了根据本公开实施例的MCTS的树扩展示意图。

[0128] 结合图12所示,一个节点中包括有节点记录和盘面,其中,盘面中涂黑的槽点表示已经扩展,盘面中未涂黑的槽点表示未扩展。对于叶子节点中,除了新特征盘面和连续算盘面,还包括节点记录,其中,该节点访问记录包括成功次数(即上述效果提升次数)、访问次数(即节点被访问次数)以及计算总数。例如,对路径上的这些数据进行加一处理。

[0129] 需要说明的是,该扩展结果为成功(success)时,则影响上述“成功次数”,或称“效果提升次数”,在某些领域还称为“胜手次数”。

[0130] 在操作S260中,基于所述节点记录和所述回报值更新所述分支路径上叶子节点的评价指标值。

[0131] 根据本公开的实施例,其中,所述叶子节点还包括节点深度,在所述基于所述节点记录和所述回报值更新所述分支路径上叶子节点的评价指标值后,还包括:判断所述扩展叶子节点的节点深度是否超过预设的深度阈值;以及在所述扩展叶子节点的节点深度满足预设的深度阈值的情况下,结束因子挖掘。

[0132] 例如,因子挖掘的终止条件是人为规定的,为了平衡计算量和计算速度,可以设置预设层数是5,那么当节点挖掘到5层时,则停止挖掘。

[0133] 在本公开的实施例中,基于研究人员发现的因子的合成有其继承性的原则,即,一个比较好的低阶因子,可能衍生出更好的高阶因子,因此,在本公开的实施例中通过蒙特卡洛树实现因子挖掘,贴合了因子在合成时具备继承性的特点,使得在基于蒙特卡洛树在生长探索的过程,可以契合因子挖掘中从低阶因子衍生出高阶因子的过程。并且蒙特卡洛树中每一叶子节点中包含的自定义的盘面设计,这改进了原有的蒙特卡洛树搜索算法,使得该新算法匹配了因子的数据特征,特别在针对规模庞大的因子空间时增强了搜索能力,该改进从算法层面解决因子空间搜索难的问题,进而提高了因子合成的有效性的同时减少了算力开销。

[0134] 研究人员还发现,因子的评价符合标准型的原则,即,当前绝大部分的因子依然是

线性因子,对高频时序数据的排序能力是其价值的直接体现。

[0135] 图3示意性示出了根据本公开实施例的仿真计算方法的流程图。

[0136] 如图3所示,该实施例的仿真计算方法包括操作S310~操作S340,该操作S310~操作S340至少可以部分执行上述操作S240。

[0137] 在操作S310中,基于所述分支路径,按照从所述根节点到所述扩展叶子节点的顺序执行因子衍生,得到因子矩阵,其中,所述因子矩阵包括多个因子数值和所述因子数值对应的因子标签。

[0138] 在操作S320中,按照所述多个因子数值的大小,通过排位矩阵算子,对所述因子矩阵中的所述因子标签执行排序,得到排序因子向量。

[0139] 具体的,根据叶子节点上的父节点引用位置逆向回溯到根节点,形成此次仿真的计算路径,从根节点开始,按照路径计算一层一层地合成因子数值。

[0140] 例如,在股票领域,所采用的根节点盘面、连续算盘面以及分别如下表五、表六以及表七如下所示:

[0141]

	最高价	最低价	收盘价	
一元算子1				
一元算子2				
.....				

[0142] 表五

[0143]

一元算子1	一元算子2	一元算子3	
			

[0144] 表六

[0145]

	最高价	最低价	收盘价	
二元算子1				
二元算子2				
.....				

[0146] 表七

[0147] 假设在扩展叶子节点合成的因子式为“A*B*C”(仅做示意,因子式可以是任意)的情况下,通过该因子式通过不同股票计算该合成式的数值,进而将这些数值的按照大小进行排名,其中,“A”对应股票的最高价或最低价或收盘价,“B”对应股票的最高价或最低价或收盘价,“C”对应股票的最高价或最低价或收盘价。对于一支股票,通过该因子式,计算出来的是一个数值;对于多组股票,通过该因子式,计算出来的是一个向量;通过该因子式,计算出来的是一个矩阵。上述因子标签则是具体某支股票的标签,通过对计算出来的因子矩阵按照数值大小进行排序,便得到股票标签的排序。

[0148] 在操作S330中,获取基准向量,所述基准向量是基准标签的排名,所述基准标签与所述因子标签一一对应。

[0149] 相应地,获取与上述因子标签对应的基准标签。例如,在股票领域,获取相对应股票的收益率,该基准向量同样是按照收益率的大小对股票进行过排名的。可以理解的是,获取的基准数据受不同的领域影响爱那个,在其他领域也可以是例如准确度、召回率等其他

的指标。

[0150] 在操作S340中,基于所述排序因子向量中的因子标签和所述基准向量中的基准标签的位差,计算所述回报值。

[0151] 例如,在股票领域,通过因子矩阵中股票的排名和基准向量中。

[0152] 总而言之,根据前面操作S310~操作S320的计算,可以合成一组股票的因子向量,根据因子向量,可以得到该组股票的排序,也就是每个股票在该组股票中的排名,也就是rank向量。根据该组股票的真实收益或称收益率,可以得到该组股票的真实排名向量,比如 $Y=[4,2,3,1,5]$ 。根据因子排名向量 X 和(或称真实排名)向量 Y ,就可以利用相关系数(例如,皮尔逊或者斯皮尔曼)的计算公式进行回报值的计算。 X_i 代表了股票 i 在因子排名向量中的名次, Y_i 代表了股票 i 在真实排名向量中的名次。

[0153] 在本公开的实施例中,通过仿真的手段,对合成因子的价值本身进行计算,保证该扩展叶子节点对应的因子式是具备较高的价值。

[0154] 考虑到算力问题,可以采用CUDA计算框架运算本公开的实施例中的因子挖掘方法,为此,设计一套MCTS-CUDA的计算框架,通过将MCTS和CUDA相结合,通过MCTS调度GPU资源,满足其在算力层面的需求。后续规范起见,一律参考CUDA编程规范,将CPU和内存硬件统称为Host,将GPU统称为Device。经过测试可以发现,该计算框架可以使得因子空间搜索的速度提升200-300倍,相比之前一年的计算量,在一两天内即可完成。

[0155] 需要说明的是,上述操作S210~操作S260均可运行在MCTS-CUDA的计算框架下,其中,操作S210、操作S220、操作S230、操作S250以及操作S260均可由Host端执行,在此不做过多赘述。

[0156] 而在操作S240中,需要由Host端和Device端通过交互实现仿真,以下将详细揭示在仿真过程中,Host和Device的交互:

[0157] 根据本公开的实施例,其中,所述基于预设的仿真逻辑,按照分支路径对所述扩展叶子节点执行仿真计算,得到回报值和扩展结果,是基于预设的CUDA计算框架执行的,其中,所述预设的CUDA计算框架包括Host端和Device端,所述基于预设的仿真逻辑,按照分支路径对所述扩展叶子节点执行仿真计算,得到回报值和扩展结果,包括:由所述CUDA计算框架,基于所述分支路径,按照从所述根节点到所述扩展叶子节点的顺序执行因子衍生,得到因子矩阵,其中,所述因子矩阵包括多个因子数值和所述因子数值对应的因子标签;由所述Device端,按照所述多个因子数值的大小,通过排位矩阵算子,对所述因子矩阵中的所述因子标签执行排序,得到排序因子向量;由所述Host端,获取基准向量,所述基准向量是基准标签的排名,所述基准标签与所述因子标签一一对应;以及由所述Device端,基于所述排序因子向量中的因子标签和所述基准向量中的基准标签的位差,计算所述回报值。

[0158] 具体的,上述操作S310仍然是采用CUDA计算框架下的Host和Device交互执行的;对于上述操作S320中进行排序时会用到的rank算子,通过Device端执行该算子;对于上述操作S330,中的基准向量,由Host端获取,再传参到Device,并在操作S340中通过Device端计算位差,得到回报值。

[0159] 图4示意性示出了根据本公开实施例的矩阵处理方法的流程图。

[0160] 如图4所示,该实施例的矩阵处理方法包括操作S410~操作S420,该操作S410~操作S420至少可以部分执行上述操作S310。

[0161] 在操作S410中,对于所述根节点盘面,通过预设的数据特征和聚合算子对收集到的高频时序数据执行降维处理,得到第一降维矩阵。

[0162] 在操作S420中,基于预设的一元算子,对所述第一降维矩阵执行计算,得到所述因子矩阵。

[0163] 具体的,在股票领域中,统计的高频时序数据是以分钟级别计的,如果在进行运算时采用最原始的高频时序数据而不进行降维的话,数据量将是巨大的。可以将分钟级的数据降维成以天计的数据,例如,对于多个股票的数据,按照上述表五、表六以及表七中,统计为股票当日的最高价、股票当日的最低价以及股票当日的收盘价。结合上述表五中的根节点盘面,对降维后的矩阵通过一元算子进行计算,得到因子矩阵。

[0164] 根据本公开的实施例,其中,所述根节点包括根节点盘面,所述由所述CUDA计算框架,基于所述分支路径,按照从所述根节点到所述扩展叶子节点的顺序执行因子衍生,得到因子矩阵,其中,所述因子矩阵包括多个因子数值和所述因子数值对应的因子标签,包括:对于所述根节点盘面,由所述Host端,通过预设的数据特征和聚合算子对收集到的高频时序数据执行降维处理,得到第一降维矩阵;以及由所述Device端,基于预设的一元算子,对所述第一降维矩阵执行计算,得到所述因子矩阵。

[0165] 具体的,在CUDA计算框架下,由于Device端的内存空间有限,故而降维的操作放置在Host端中完成,形成降维矩阵(因为降维可能性有限,可缓存Host端,在Device端需要计算相关数值的时候拷贝,以便重复使用),因此,上述操作S410由Host端执行。而对于操作S420,将降维矩阵通过一元算子进行计算的操作便防止在计算能力更强的Device端进行。

[0166] 图5示意性示出了根据本公开实施例的另一种矩阵处理方法的流程图。

[0167] 如图5所示,该实施例的另一种矩阵处理方法包括操作S510~操作S530,该操作S510~操作S530至少可以部分执行上述操作S310。

[0168] 在操作S510中,对于所述连续算盘面,通过预设的一元算子,对上一叶子节点生成的因子矩阵执行计算,得到新的因子矩阵。

[0169] 在操作S520中,对于所述连续算盘面,通过预设的数据特征和聚合算子对收集到的高频时序数据执行降维处理,得到第二降维矩阵。

[0170] 在操作S530中,基于预设的二元算子,对所述第二降维矩阵和上一叶子节点生成的因子矩阵执行计算,得到新的因子矩阵。

[0171] 具体的,对于叶子节点,采用叶子节点预先设定好的连续算盘面和新特征盘面对上一节点生成的因子矩阵进行因子合成处理,其中,对于新特征盘面,还需要引入新的数据参与计算。

[0172] 根据本公开的实施例,其中,所述叶子节点包括连续算盘面和新特征盘面,所述由所述CUDA计算框架,基于所述分支路径,按照从所述根节点到所述扩展叶子节点的顺序执行因子衍生,得到因子矩阵,其中,所述因子矩阵包括多个因子数值和所述因子数值对应的因子标签,还包括:对于所述连续算盘面,由所述Device端,通过预设的一元算子,对上一叶子节点生成的因子矩阵执行计算,得到新的因子矩阵;对于所述连续算盘面,由所述Host端,通过预设的数据特征和聚合算子对收集到的高频时序数据执行降维处理,得到第二降维矩阵;以及由所述Device端,基于预设的二元算子,对所述第二降维矩阵和上一叶子节点生成的因子矩阵执行计算,得到新的因子矩阵。

[0173] 具体的,与上述操作S410~操作S420同一原理,对原始的高频时序数据进行降维操作(或称group操作)放置在Host端进行,以充分利用Host端的内存资源,一元算子和二元算子的计算,放置在Device端进行,以充分利用Device端的算力优势,同时避免Device端GPU中内存较小的缺陷。

[0174] 图6示意性示出了根据本公开实施例的回报值计算方法的流程图。

[0175] 如图6所示,该实施例的回报值计算方法包括操作S610~操作S620,该操作S610~操作S620至少可以部分执行上述操作S340。

[0176] 在操作S610中,获取所述因子矩阵的数据量或者所述基准向量的数据量。

[0177] 在操作S620中,通过所述位差和所述数据量,计算所述回报值。

[0178] 具体的,可以通过spearman相关系数作为回报值的计算式,如下式一所示:

$$[0179] \quad reward = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)} \quad \text{式一}$$

[0180] 其中,n表示数据量, d_i 表示 X_i 和 Y_i 的位次差,i代表某个数据。

[0181] 根据本公开的实施例,其中,所述由所述Device端,基于所述排序因子向量中的因子标签和所述基准向量中的基准标签的位差,计算所述回报值,包括:由所述Device端,获取所述因子矩阵的数据量或者所述基准向量的数据量;以及由所述Device端,通过所述位差和所述数据量,计算所述回报值。

[0182] 具体的,除了上述一元算子和二元算子由Device端执行,计算回报值所用到的spearman算子也由Device端执行。

[0183] 图7示意性示出了根据本公开实施例的另一种仿真计算方法的流程图。

[0184] 如图7所示,该实施例的另一种仿真计算方法包括操作S710~操作S720,该操作S710~操作S720至少可以部分执行上述操作S240。

[0185] 在操作S710中,判断所述扩展叶子节点是否符合预设的扩展成功条件,所述预设的扩展成功条件包括多个扩展子条件,所述多个扩展子条件用以确定所述扩展叶子节点的可挖掘性。

[0186] 在操作S720中,在所述扩展叶子节点符合所述多个扩展子条件中任意一个的情况下,判定所述扩展结果为成功。

[0187] 可以理解的是,除了对扩展叶子节点对应的因子式的价值进行计算外,还需要确定扩展叶子节点的可挖掘性,以保证通过该维度的扩展叶子节点,可以进一步实现更高维的因子式合成。也就是说,仿真计算还对该节点的可扩展性/可挖掘性进行计算,保证朝向该扩展叶子节点的方向进行因子挖掘的挖掘前景是正向的。

[0188] 根据本公开的实施例,其中,所述基于预设的仿真逻辑,按照分支路径对所述扩展叶子节点执行仿真计算,得到回报值和扩展结果,还包括:由所述CUDA计算框架,判断所述扩展叶子节点是否符合预设的扩展成功条件,所述预设的扩展成功条件包括多个扩展子条件,所述多个扩展子条件用以确定所述扩展叶子节点的可挖掘性;以及由所述CUDA计算框架,在所述扩展叶子节点符合所述多个扩展子条件中任意一个的情况下,判定所述扩展结果为成功。

[0189] 具体的,上述操作S710和上述操作S720都是由Host端或者由Device端单独或交互着执行的。

[0190] 图8示意性示出了根据本公开实施例的扩展子条件的验证方法的流程图。

[0191] 如图8所示,该实施例的扩展子条件的验证方法包括操作S810~操作S820,该操作S810~操作S820至少可以部分执行上述操作S710。

[0192] 在操作S810中,判断所述扩展叶子节点的回报值是否处于预设回报值区间。

[0193] 在操作S820中,在所述扩展叶子节点的回报值处于所述预设回报值区间内的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0194] 根据本公开的实施例,其中,所述多个扩展子条件包括的验证方法包括:由所述Host端,判断所述扩展叶子节点的回报值是否处于预设回报值区间;以及由所述Host端,在所述扩展叶子节点的回报值处于所述预设回报值区间内的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0195] 具体的,在CUDA计算框架下,回报值保存在叶子节点上,因此,上述操作S810~操作S820可以直接由Host端执行,通过判断回报值(reward)是否满足一定的阈值,满足则认为扩展结果为成功。

[0196] 图9示意性示出了根据本公开实施例的另一种扩展子条件的验证方法的流程图。

[0197] 如图9所示,该实施例的另一种扩展子条件的验证方法包括操作S910~操作S930,该操作S910~操作S930至少可以部分执行上述操作S710。

[0198] 在操作S910中,基于所述分支路径,按照从所述扩展叶子节点到所述根节点的顺序,回溯预设的个数的叶子节点的回报值。

[0199] 在操作S920中,基于所述扩展叶子节点的回报值和所述预设的个数的叶子节点的回报值,判断所述回报值是否处于增长趋势。

[0200] 在操作S930中,在所述回报值处于增长趋势的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0201] 研究人员发现,在某个方向上连续几轮探测没有显著提升的因子合成,大概率不会存在较好的因子可能性,因此,对于某些叶子节点对应的因子式回报值可能较低,但是可挖掘性较强的,回溯该条路径上的多个父节点的回报值,判断该回报值的总体趋势是否上升。

[0202] 根据本公开的实施例,其中,所述多个扩展子条件包括的验证方法还包括:由所述Host端,基于所述分支路径,按照从所述扩展叶子节点到所述根节点的顺序,回溯预设的个数的叶子节点的回报值;由所述Host端,基于所述扩展叶子节点的回报值和所述预设的个数的叶子节点的回报值,判断所述回报值是否处于增长趋势;以及由所述Host端,在所述回报值处于增长趋势的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0203] 具体的,在CUDA计算框架下,由于历史的回报值存储在叶子节点上,因此,无需调用复杂的算子,直接获取和校验多个回报值是否呈增长趋势即可,具体的,可以按照从上至下的顺序判断叶子节点对应的因子合成式的回报值是否一直增加。综上,上述操作S910~操作S930均由Host端执行。

[0204] 图10示意性示出了根据本公开实施例的扩展子条件的验证方法的流程图。

[0205] 如图10所示,该实施例的扩展子条件的验证方法包括操作S1010~操作S1030,该操作S1010~操作S1030至少可以部分执行上述操作S710。

[0206] 在操作S1010中,基于快速走步算子,从所述扩展叶子节点向下随机探索预设步

数,得到所述扩展叶子节点下层路径的多个叶子节点的回报值。

[0207] 在操作S1020中,基于所述扩展叶子节点的回报值和所述扩展叶子节点下层路径的多个叶子节点,判断所述回报值是否处于增长趋势。

[0208] 在操作S1030中,在所述回报值处于增长趋势的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0209] 研究人员发现,在某个方向上连续几轮探测没有显著提升的因子合成,大概率不会存在较好的因子可能性,因此,对于某些叶子节点对应的因子式回报值可能较低,但是可挖掘性较强的,回溯该条路径上的多个父节点的回报值,判断该回报值的总体趋势是否上升。

[0210] 其中,上述快速走步算子的核心思想是:从该扩展叶子节点开始,随机向下走n步,看看reward是否整体有增加的趋势。

[0211] 根据本公开的实施例,其中,所述多个扩展子条件包括的验证方法还包括:由所述Device端,基于快速走步算子,从所述扩展叶子节点向下随机探索预设步数,得到所述扩展叶子节点下层路径的多个叶子节点的回报值;由所述Host端,基于所述扩展叶子节点的回报值和所述扩展叶子节点下层路径的多个叶子节点,判断所述回报值是否处于增长趋势;以及由所述Host端,在所述回报值处于增长趋势的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0212] 具体的,在CUDA计算框架下,上述操作S1010的快速走步算子由Device端执行,上述操作S1020和操作S1030由Host端执行。

[0213] 图11示意性示出了根据本公开实施例的评价指标值更新方法的流程图。

[0214] 如图11所示,该实施例的评价指标值更新方法包括操作S1110~操作S1130,该操作S1110~操作S1130至少可以部分执行上述操作S260。

[0215] 在操作S1110中,基于所述效果提升次数和所述节点被访问次数,计算所述叶子节点的开发能力值。

[0216] 具体的,开发能力值的计算方式如下式二所示:

$$[0217] \quad \text{exploitation} = \frac{\text{success}}{\text{visit}} \quad \text{式二}$$

[0218] 其中,success指效果提升次数,visit指节点被访问次数。表示评价该分支挖掘出特异因子的可能性,主要是依据因子合成的继承性,评判该分支继续搜索的前途如何。

[0219] 在操作S1120中,基于所述节点被访问次数和所述计算总数,计算所述叶子节点的勘探能力值。

[0220] 具体的,勘探能力值的计算方式如下式三所示:

$$[0221] \quad \text{exploration} = \sqrt{\frac{\ln(t)}{\text{visit}}} \quad \text{式三}$$

[0222] 其中,t指计算总数,visit指节点被访问次数。表示该节点应该被探测的分支,从物理意义上来说,就是当其他节点的探查并没有带来额外的开发能力时,之前探测较少的节点将会得到更高的分数。

[0223] 在操作S1130中,基于所述回报值、所述开发能力值以及所述勘探能力值,通过加权的方式执行计算,得到所述评价指标值。

[0224] 根据本公开的实施例,其中,回报值用以评价所述叶子节点的深度,所述开发能力值用以评价所述叶子节点的深度,所述勘探能力值用以评价所述叶子节点的广度。

[0225] 具体的,评价指标值如下式四所述:

[0226] $UCB = [\text{reward}, \text{exploitation}, \text{exploration}] \times [\omega_1, \omega_2, \omega_3]^T$ 式四

[0227] 其中,UCB是指评价指标值, $\omega_1, \omega_2, \omega_3$ 分别是三个超参数,框架使用人员可以根据计算的情况和效果,进行灵活调整。比如说reward的数量级是0.001,而exploitation的数量级是0.1,那么reward可能就起不到作用,这时候给reward新增一个100的 ω 超参数,就可以让reward的作用和exploitation在同一个数量级上。

[0228] 在本公开的实施例中,评价指标值通过不同权重可以兼顾因子挖掘过程中生成的叶子节点的深度和广度。保证更多具备价值的合成因子式被生成。

[0229] 图13示意性示出了根据本公开实施例的CUDA计算架构下Host端和Device端的调度示意图。

[0230] 如图13所示,MCTS的算法调度在Host端完成,而因子的合成和评价指标计算在Device端中完成。Group算子的操作均在Host端执行,不同的一元算子、二元算子的执行均在Device端执行,评价指标(图13中的评价指标即是上述回报值)也在Device端执行。并且,涉及不同节点的盘面中槽点的操作也在Host端执行。

[0231] 当然,在执行group操作之前,还可以使用空值识别算子剔除高频时序数据中存在空值的不合理的数据。由于处理的数据量较大,因此,对于这类空值识别算子,其执行端可以设置于Host端中。

[0232] 基于上述因子挖掘方法,本公开还提供了一种因子挖掘装置。以下将结合图14对该装置进行详细描述。

[0233] 图14示意性示出了根据本公开实施例的因子挖掘装置的结构框图。

[0234] 如图14所示,该实施例的因子挖掘装置1400包括数据获取模块1410、节点选择模块1420、节点扩展模块1430、仿真计算模块1440、反向传播模块1450和评价指标更新模块1460。

[0235] 根据本公开的实施例,其中,所述因子挖掘装置由蒙特卡洛树实现,所述蒙特卡洛树包括一个根节点和多个叶子节点,所述叶子节点包括盘面和节点记录,所述盘面包括算子和特征,所述盘面包括多个槽点,所述槽点包括已扩展和未扩展。

[0236] 数据获取模块1410用于获取当前树结构和所述当前树结构中叶子节点的评价指标值,其中,所述当前树结构的叶子节点包括第一叶子节点,所述第一叶子节点的盘面包括未扩展的槽点。在一实施例中,数据获取模块1410可以用于执行前文描述的操作S210,在此不再赘述。

[0237] 节点选择模块1420用于选择所述评价指标值最大的第一叶子节点作为待扩展叶子节点。在一实施例中,节点选择模块1420可以用于执行前文描述的操作S220,在此不再赘述。

[0238] 节点扩展模块1430用于选取所述待扩展叶子节点上的盘面的未扩展的槽点执行节点扩展,得到扩展叶子节点。在一实施例中,节点扩展模块1430可以用于执行前文描述的操作S230,在此不再赘述。

[0239] 仿真计算模块1440用于基于预设的仿真逻辑,按照分支路径对所述扩展叶子节点

执行仿真计算,得到回报值和扩展结果,其中,所述分支路径是从所述根节点到所述扩展叶子节点的路径。在一实施例中,仿真计算模块1440可以用于执行前文描述的操作S240,在此不再赘述。

[0240] 反向传播模块1450用于在所述扩展结果为扩展成功的情况下,基于所述分支路径执行反向传播操作,更新所述分支路径上根节点和叶子节点的节点记录。在一实施例中,反向传播模块1450可以用于执行前文描述的操作S250,在此不再赘述。

[0241] 评价指标更新模块1460用于基于所述节点记录和所述回报值更新所述分支路径上叶子节点的评价指标值。在一实施例中,评价指标更新模块1460可以用于执行前文描述的操作S260,在此不再赘述。

[0242] 在本公开的实施例中,基于研究人员发现的因子的合成有其继承性的原则,即,一个比较好的低阶因子,可能衍生出更好的高阶因子,因此,在本公开的实施例中通过蒙特卡洛树实现因子挖掘,贴合了因子在合成时具备继承性的特点,使得在基于蒙特卡洛树在生长探索的过程,可以契合因子挖掘中从低阶因子衍生出高阶因子的过程。并且蒙特卡洛树中每一叶子节点中包含的自定义的盘面设计,这改进了原有的蒙特卡洛树搜索算法,使得该新算法匹配了因子的数据特征,特别在针对规模庞大的因子空间时增强了搜索能力,该改进从算法层面解决因子空间搜索难的问题,进而提高了因子合成的有效性的同时减少了算力开销。

[0243] 根据本公开的实施例,其中,所述仿真计算模块,还用于基于所述分支路径,按照从所述根节点到所述扩展叶子节点的顺序执行因子衍生,得到因子矩阵,其中,所述因子矩阵包括多个因子数值和所述因子数值对应的因子标签;按照所述多个因子数值的大小,通过排位矩阵算子,对所述因子矩阵中的所述因子标签执行排序,得到排序因子向量;获取基准向量,所述基准向量是基准标签的排名,所述基准标签与所述因子标签一一对应;以及基于所述排序因子向量中的因子标签和所述基准向量中的基准标签的位差,计算所述回报值。

[0244] 根据本公开的实施例,其中,所述根节点包括根节点盘面,所述仿真计算模块,还用于对于所述根节点盘面,通过预设的数据特征和聚合算子对收集到的高频时序数据执行降维处理,得到第一降维矩阵;以及基于预设的一元算子,对所述第一降维矩阵执行计算,得到所述因子矩阵。

[0245] 根据本公开的实施例,其中,所述仿真计算模块,还用于对于所述连续算盘面,通过预设的一元算子,对上一叶子节点生成的因子矩阵执行计算,得到新的因子矩阵;对于所述连续算盘面,通过预设的数据特征和聚合算子对收集到的高频时序数据执行降维处理,得到第二降维矩阵;以及基于预设的二元算子,对所述第二降维矩阵和上一叶子节点生成的因子矩阵执行计算,得到新的因子矩阵。

[0246] 根据本公开的实施例,其中,所述仿真计算模块,还用于获取所述因子矩阵的数据量或者所述基准向量的数据量;以及通过所述位差和所述数据量,计算所述回报值。

[0247] 根据本公开的实施例,其中,所述仿真计算模块,还用于判断所述扩展叶子节点是否符合预设的扩展成功条件,所述预设的扩展成功条件包括多个扩展子条件,所述多个扩展子条件用以确定所述扩展叶子节点的可挖掘性;以及在所述扩展叶子节点符合所述多个扩展子条件中任意一个的情况下,判定所述扩展结果为成功。

[0248] 根据本公开的实施例,其中,所述仿真计算模块,还用于判断所述扩展叶子节点的回报值是否处于预设回报值区间;以及在所述扩展叶子节点的回报值处于所述预设回报值区间内的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0249] 根据本公开的实施例,其中,所述仿真计算模块,还用于基于所述分支路径,按照从所述扩展叶子节点到所述根节点的顺序,回溯预设的个数的叶子节点的回报值;基于所述扩展叶子节点的回报值和所述预设的个数的叶子节点的回报值,判断所述回报值是否处于增长趋势;以及在所述回报值处于增长趋势的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0250] 根据本公开的实施例,其中,所述仿真计算模块,还用于基于快速走步算子,从所述扩展叶子节点向下随机探索预设步数,得到所述扩展叶子节点下层路径的多个叶子节点的回报值;基于所述扩展叶子节点的回报值和所述扩展叶子节点下层路径的多个叶子节点,判断所述回报值是否处于增长趋势;以及在所述回报值处于增长趋势的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0251] 根据本公开的实施例,其中,所述仿真计算模块,是基于预设的CUDA计算框架执行的,其中,所述预设的CUDA计算框架包括Host端和Device端,所述仿真计算模块,还用于由所述CUDA计算框架,基于所述分支路径,按照从所述根节点到所述扩展叶子节点的顺序执行因子衍生,得到因子矩阵,其中,所述因子矩阵包括多个因子数值和所述因子数值对应的因子标签;由所述Device端,按照所述多个因子数值的大小,通过排位矩阵算子,对所述因子矩阵中的所述因子标签执行排序,得到排序因子向量;由所述Host端,获取基准向量,所述基准向量是基准标签的排名,所述基准标签与所述因子标签一一对应;以及由所述Device端,基于所述排序因子向量中的因子标签和所述基准向量中的基准标签的位差,计算所述回报值。

[0252] 根据本公开的实施例,其中,所述根节点包括根节点盘面,所述仿真计算模块,还用于对于所述根节点盘面,由所述Host端,通过预设的数据特征和聚合算子对收集到的高频时序数据执行降维处理,得到第一降维矩阵;以及由所述Device端,基于预设的一元算子,对所述第一降维矩阵执行计算,得到所述因子矩阵。

[0253] 根据本公开的实施例,其中,所述仿真计算模块,还用于对于所述连续算盘面,由所述Device端,通过预设的一元算子,对上一叶子节点生成的因子矩阵执行计算,得到新的因子矩阵;对于所述连续算盘面,由所述Host端,通过预设的数据特征和聚合算子对收集到的高频时序数据执行降维处理,得到第二降维矩阵;以及由所述Device端,基于预设的二元算子,对所述第二降维矩阵和上一叶子节点生成的因子矩阵执行计算,得到新的因子矩阵。

[0254] 根据本公开的实施例,其中,所述仿真计算模块,还用于由所述Device端,获取所述因子矩阵的数据量或者所述基准向量的数据量;以及由所述Device端,通过所述位差和所述数据量,计算所述回报值。

[0255] 根据本公开的实施例,其中,所述仿真计算模块,还用于由所述CUDA计算框架,判断所述扩展叶子节点是否符合预设的扩展成功条件,所述预设的扩展成功条件包括多个扩展子条件,所述多个扩展子条件用以确定所述扩展叶子节点的可挖掘性;以及由所述CUDA计算框架,在所述扩展叶子节点符合所述多个扩展子条件中任意一个的情况下,判定所述扩展结果为成功。

[0256] 根据本公开的实施例,其中,所述仿真计算模块,还用于由所述Host端,判断所述扩展叶子节点的回报值是否处于预设回报值区间;以及由所述Host端,在所述扩展叶子节点的回报值处于所述预设回报值区间内的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0257] 根据本公开的实施例,其中,所述仿真计算模块,还用于由所述Host端,基于所述分支路径,按照从所述扩展叶子节点到所述根节点的顺序,回溯预设的个数的叶子节点的回报值;由所述Host端,基于所述扩展叶子节点的回报值和所述预设的个数的叶子节点的回报值,判断所述回报值是否处于增长趋势;以及由所述Host端,在所述回报值处于增长趋势的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0258] 根据本公开的实施例,其中,所述仿真计算模块,还用于由所述Device端,基于快速走步算子,从所述扩展叶子节点向下随机探索预设步数,得到所述扩展叶子节点下层路径的多个叶子节点的回报值;由所述Host端,基于所述扩展叶子节点的回报值和所述扩展叶子节点下层路径的多个叶子节点,判断所述回报值是否处于增长趋势;以及由所述Host端,在所述回报值处于增长趋势的情况下,确定所述扩展叶子节点符合所述扩展子条件。

[0259] 根据本公开的实施例,其中,所述节点记录包括:效果提升次数、节点被访问次数以及计算总数,所述反向传播模块,还用于更新所述分支路径上根节点和叶子节点的所述效果提升次数、所述节点被访问次数以及所述计算总数。

[0260] 根据本公开的实施例,其中,所述评价指标更新模块,还用于基于所述效果提升次数和所述节点被访问次数,计算所述叶子节点的开发能力值,所述开发能力值用以评价所述叶子节点的深度;基于所述节点被访问次数和所述计算总数,计算所述叶子节点的勘探能力值,所述勘探能力值用以评价所述叶子节点的广度;以及基于所述回报值、所述开发能力值以及所述勘探能力值,通过加权的方式执行计算,得到所述评价指标值。

[0261] 根据本公开的实施例,其中,所述节点选择模块,还用于在多个所述评价指标值相同的情况下,随机挑选第一叶子节点作为待扩展叶子节点。

[0262] 根据本公开的实施例,其中,所述节点扩展模块,还用于通过加权抽取,随机对所述新特征盘面和所述连续算盘面中未扩展的槽点执行抽取;以及基于抽取到的未扩展的槽点执行节点扩展,得到扩展叶子节点。

[0263] 根据本公开的实施例,其中,所述叶子节点还包括节点深度,所述装置还包括截止模块,用于判断所述扩展叶子节点的节点深度是否超过预设的深度阈值;以及在所述扩展叶子节点的节点深度满足预设的深度阈值的情况下,结束因子挖掘。

[0264] 根据本公开的实施例,获取模块1410、节点选择模块1420、节点扩展模块1430、仿真计算模块1440、反向传播模块1450和评价指标更新模块1460中的任意多个模块可以合并在一个模块中实现,或者其中的任意一个模块可以被拆分成多个模块。或者,这些模块中的一个或多个模块的至少部分功能可以与其他模块的至少部分功能相结合,并在一个模块中实现。根据本公开的实施例,获取模块1410、节点选择模块1420、节点扩展模块1430、仿真计算模块1440、反向传播模块1450和评价指标更新模块1460中的至少一个可以至少被部分地实现为硬件电路,例如现场可编程门阵列(FPGA)、可编程逻辑阵列(PLA)、片上系统、基板上的系统、封装上的系统、专用集成电路(ASIC),或可以通过对电路进行集成或封装的任何其他的合理方式等硬件或固件来实现,或以软件、硬件以及固件三种实现方式中任意一种或

以其中任意几种的适当组合来实现。或者,获取模块1410、节点选择模块1420、节点扩展模块1430、仿真计算模块1440、反向传播模块1450和评价指标更新模块1460中的至少一个可以至少被部分地实现为计算机程序模块,当该计算机程序模块被运行时,可以执行相应的功能。

[0265] 图15示意性示出了根据本公开实施例的适于实现因子挖掘方法的电子设备的方框图。

[0266] 如图15所示,根据本公开实施例的电子设备1500包括处理器1501,其可以根据存储在只读存储器 (ROM) 1502中的程序或者从存储部分1508加载到随机访问存储器 (RAM) 1503中的程序而执行各种适当的动作和处理。处理器1501例如可以包括通用微处理器(例如CPU)、指令集处理器和/或相关芯片组和/或专用微处理器(例如,专用集成电路(ASIC))等等。处理器1501还可以包括用于缓存用途的板载存储器。处理器1501可以包括用于执行根据本公开实施例的方法流程的不同动作的单一处理单元或者是多个处理单元。

[0267] 在RAM 1503中,存储有电子设备1500操作所需的各种程序和数据。处理器1501、ROM 1502以及RAM 1503通过总线1504彼此相连。处理器1501通过执行ROM 1502和/或RAM 1503中的程序来执行根据本公开实施例的方法流程的各种操作。需要注意,所述程序也可以存储在除ROM 1502和RAM 1503以外的一个或多个存储器中。处理器1501也可以通过执行存储在所述一个或多个存储器中的程序来执行根据本公开实施例的方法流程的各种操作。

[0268] 根据本公开的实施例,电子设备1500还可以包括输入/输出(I/O)接口1505,输入/输出(I/O)接口1505也连接至总线1504。电子设备1500还可以包括连接至I/O接口1505的以下部件中的一项或多项:包括键盘、鼠标等的输入部分1506;包括诸如阴极射线管(CRT)、液晶显示器(LCD)等以及扬声器等的输出部分1507;包括硬盘等的存储部分1508;以及包括诸如LAN卡、调制解调器等的网络接口卡的通信部分1509。通信部分1509经由诸如因特网的网络执行通信处理。驱动器1510也根据需要连接至I/O接口1505。可拆卸介质1511,诸如磁盘、光盘、磁光盘、半导体存储器等等,根据需要安装在驱动器1510上,以便于从其上读出的计算机程序根据需要被安装入存储部分1508。

[0269] 本公开还提供了一种计算机可读存储介质,该计算机可读存储介质可以是上述实施例中描述的设备/装置/系统中所包含的;也可以是单独存在,而未装配入该设备/装置/系统中。上述计算机可读存储介质承载有一个或者多个程序,当上述一个或者多个程序被执行时,实现根据本公开实施例的方法。

[0270] 根据本公开的实施例,计算机可读存储介质可以是非易失性的计算机可读存储介质,例如可以包括但不限于:便携式计算机磁盘、硬盘、随机访问存储器(RAM)、只读存储器(ROM)、可擦式可编程只读存储器(EPROM或闪存)、便携式紧凑磁盘只读存储器(CD-ROM)、光存储器件、磁存储器件、或者上述的任意合适的组合。在本公开中,计算机可读存储介质可以是任何包含或存储程序的有形介质,该程序可以被指令执行系统、装置或者器件使用或者与其结合使用。例如,根据本公开的实施例,计算机可读存储介质可以包括上文描述的ROM 1502和/或RAM 1503和/或ROM 1502和RAM 1503以外的一个或多个存储器。

[0271] 本公开的实施例还包括一种计算机程序产品,其包括计算机程序,该计算机程序包含用于执行流程图所示的方法的程序代码。当计算机程序产品在计算机系统中运行时,该程序代码用于使计算机系统实现本公开实施例所提供的方法。

[0272] 在该计算机程序被处理器1501执行时执行本公开实施例的系统/装置中限定的上述功能。根据本公开的实施例,上文描述的系统、装置、模块、单元等可以通过计算机程序模块来实现。

[0273] 在一种实施例中,该计算机程序可以依托于光存储器件、磁存储器件等有形存储介质。在另一种实施例中,该计算机程序也可以在网络介质上以信号的形式进行传输、分发,并通过通信部分1509被下载和安装,和/或从可拆卸介质1511被安装。该计算机程序包含的程序代码可以用任何适当的网络介质传输,包括但不限于:无线、有线等等,或者上述的任意合适的组合。

[0274] 在这样的实施例中,该计算机程序可以通过通信部分1509从网络上被下载和安装,和/或从可拆卸介质1511被安装。在该计算机程序被处理器1501执行时,执行本公开实施例的系统中限定的上述功能。根据本公开的实施例,上文描述的系统、设备、装置、模块、单元等可以通过计算机程序模块来实现。

[0275] 根据本公开的实施例,可以以一种或多种程序设计语言的任意组合来编写用于执行本公开实施例提供的计算机程序的程序代码,具体地,可以利用高级过程和/或面向对象的编程语言、和/或汇编/机器语言来实施这些计算程序。程序设计语言包括但不限于诸如Java,C++,python,“C”语言或类似的程序设计语言。程序代码可以完全地在用户计算设备上执行、部分地在用户设备上执行、部分在远程计算设备上执行、或者完全在远程计算设备或服务器上执行。在涉及远程计算设备的情形中,远程计算设备可以通过任意种类的网络,包括局域网(LAN)或广域网(WAN),连接到用户计算设备,或者,可以连接到外部计算设备(例如利用因特网服务提供商来通过因特网连接)。

[0276] 附图中的流程图和框图,图示了按照本公开各种实施例的系统、方法和计算机程序产品的可能实现的体系架构、功能和操作。在这点上,流程图或框图中的每个方框可以代表一个模块、程序段、或代码的一部分,上述模块、程序段、或代码的一部分包含一个或多个用于实现规定的逻辑功能的可执行指令。也应当注意,在有些作为替换的实现中,方框中所标注的功能也可以以不同于附图中所标注的顺序发生。例如,两个接连地表示的方框实际上可以基本并行地执行,它们有时也可以按相反的顺序执行,这依所涉及的功能而定。也要注意,框图或流程图中的每个方框、以及框图或流程图中的方框的组合,可以用执行规定的功能或操作的专用的基于硬件的系统来实现,或者可以用专用硬件与计算机指令的组合来实现。

[0277] 本领域技术人员可以理解,本公开的各个实施例和/或权利要求中记载的特征可以进行多种组合和/或结合,即使这样的组合或结合没有明确记载于本公开中。特别地,在不脱离本公开精神和教导的情况下,本公开的各个实施例和/或权利要求中记载的特征可以进行多种组合和/或结合。所有这些组合和/或结合均落入本公开的范围。

[0278] 以上对本公开的实施例进行了描述。但是,这些实施例仅仅是为了说明的目的,而并非为了限制本公开的范围。尽管在以上分别描述了各实施例,但是这并不意味着各个实施例中的措施不能有利地结合使用。本公开的范围由所附权利要求及其等同物限定。不脱离本公开的范围,本领域技术人员可以做出多种替代和修改,这些替代和修改都应落在本公开的范围之内。

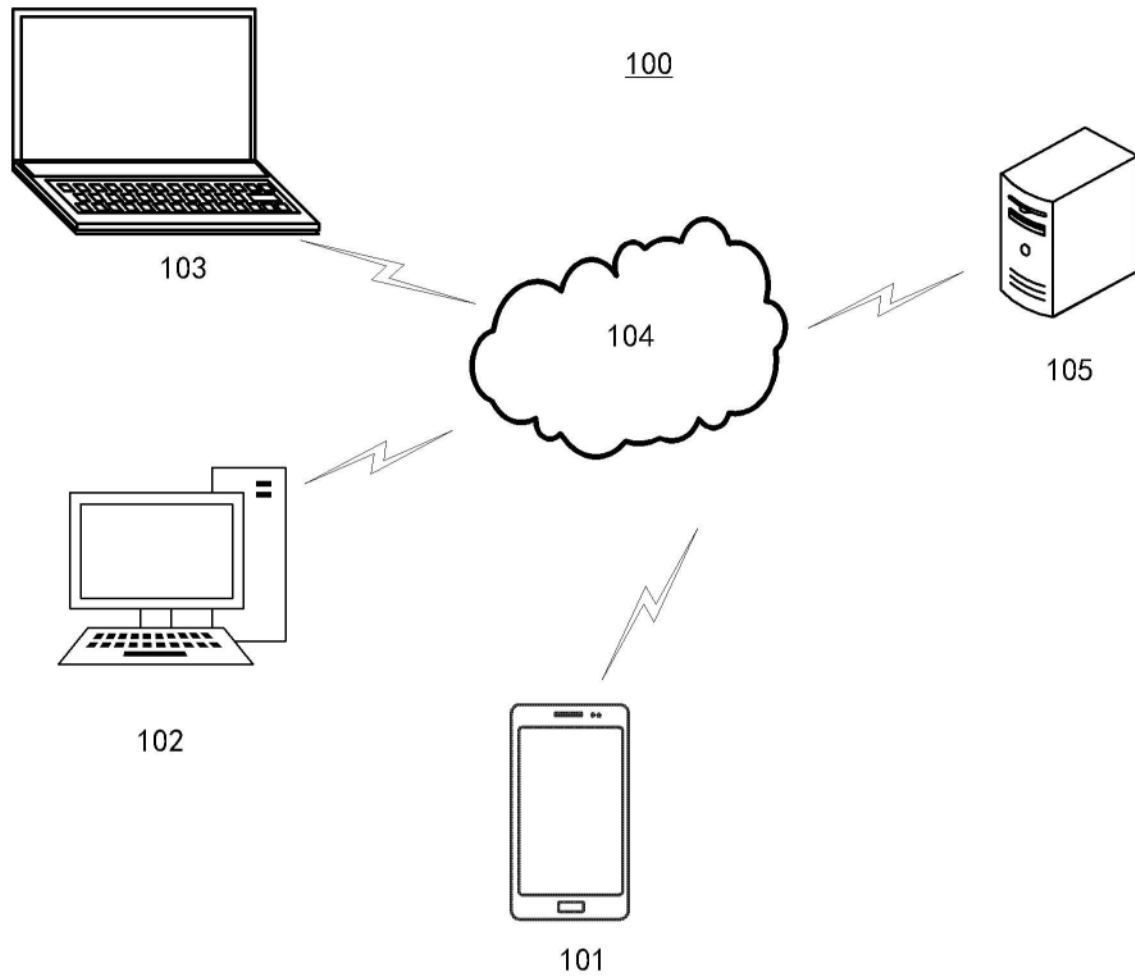


图1

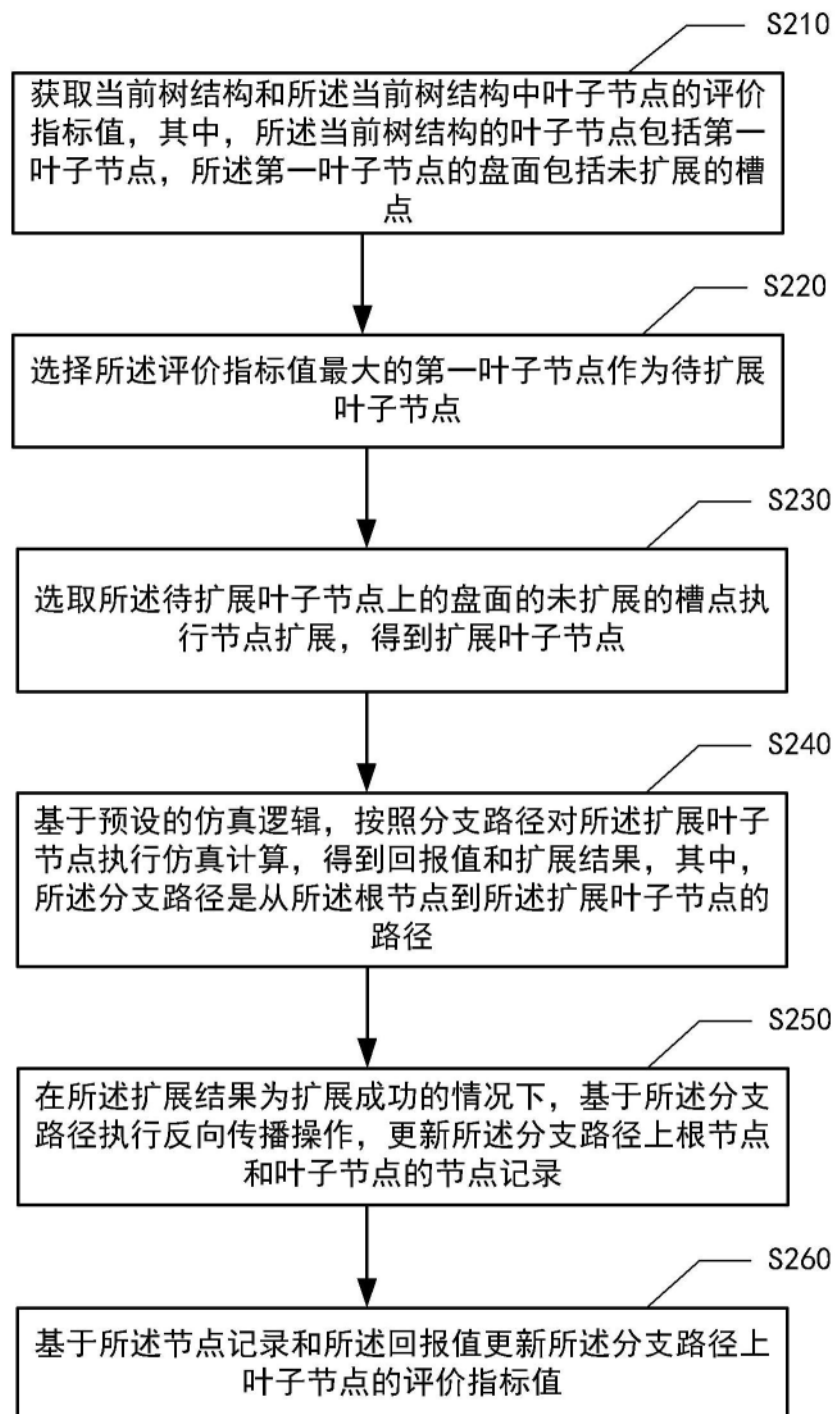


图2

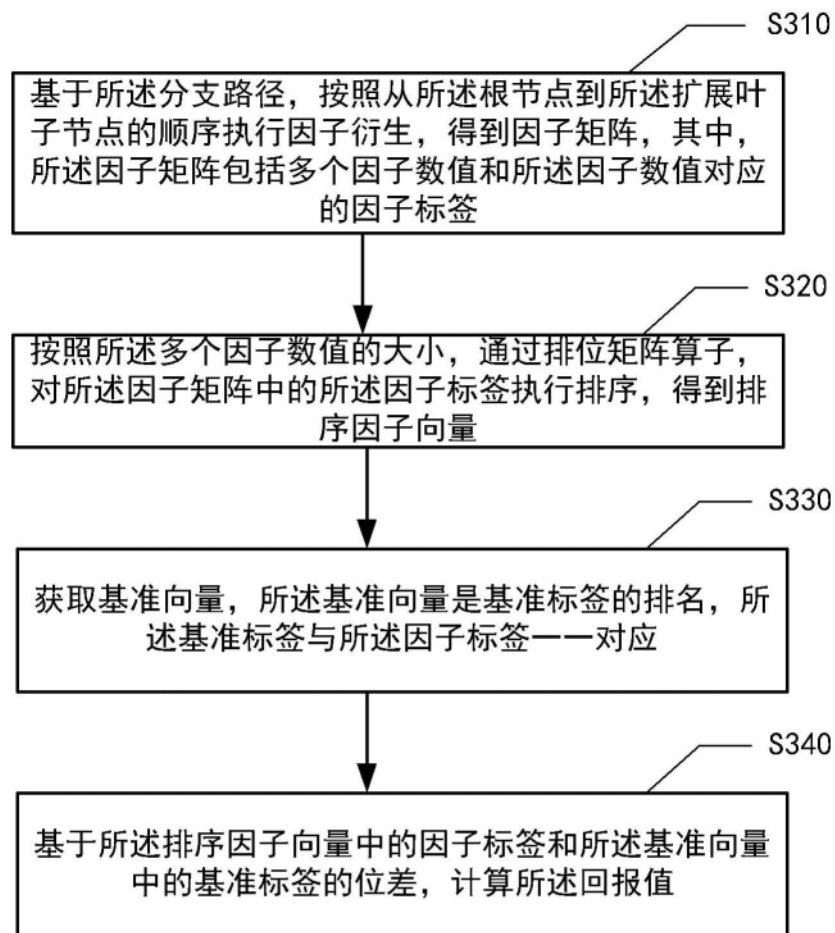


图3

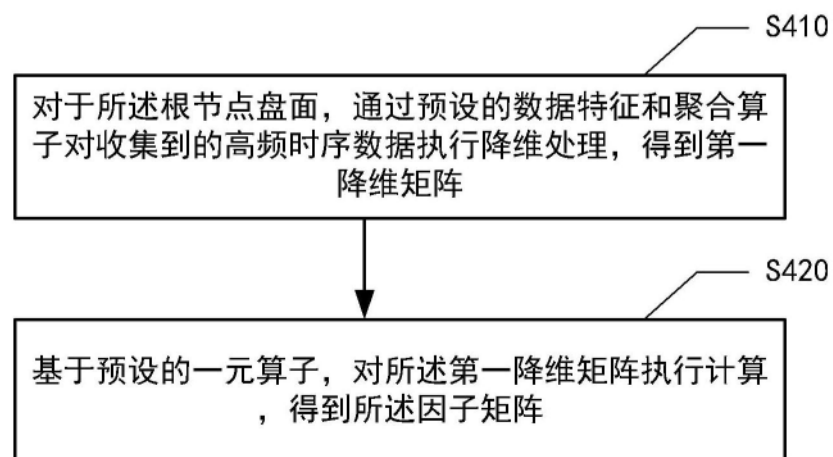


图4

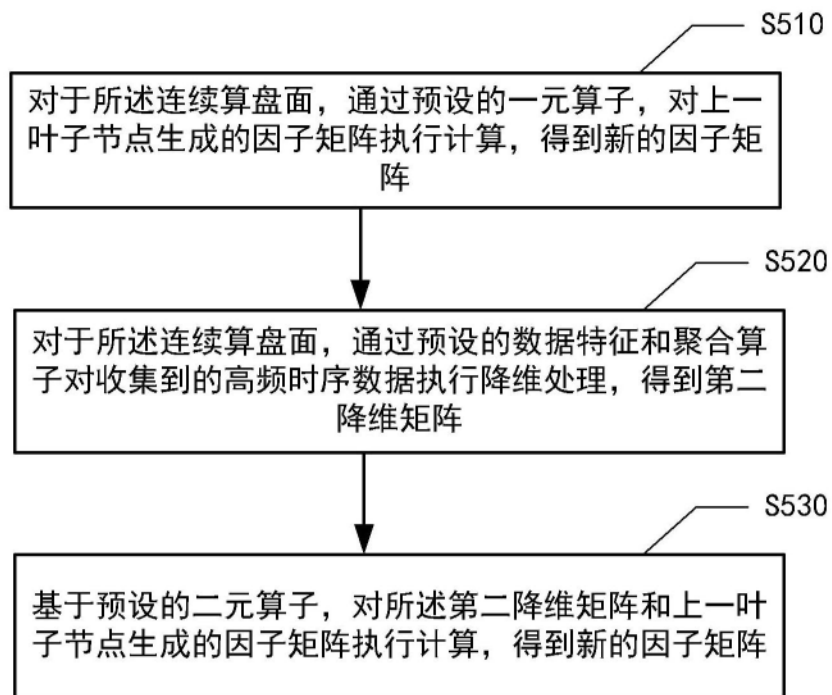


图5

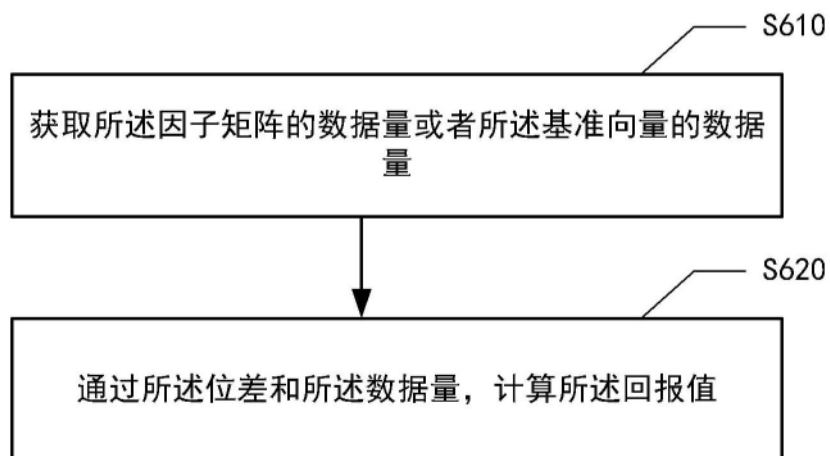


图6

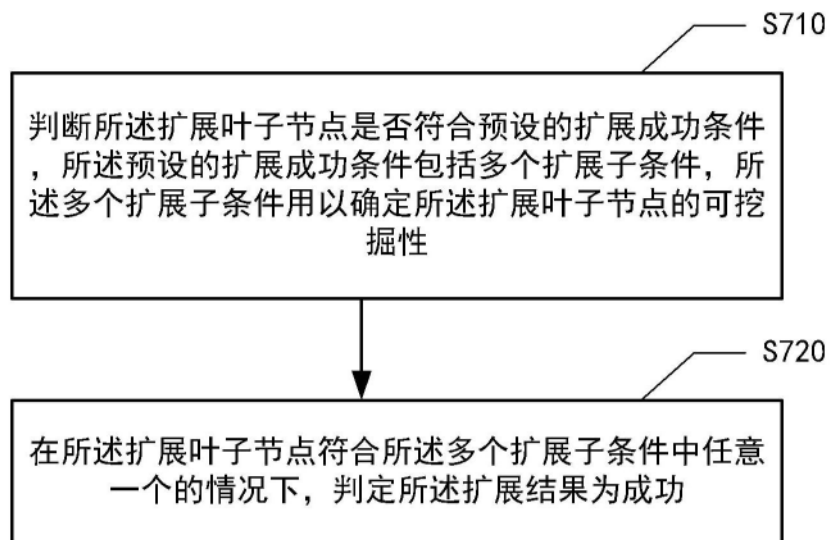


图7

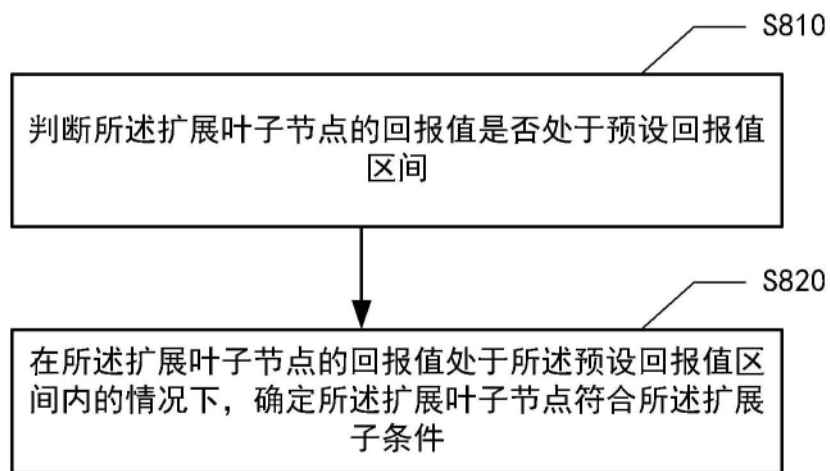


图8

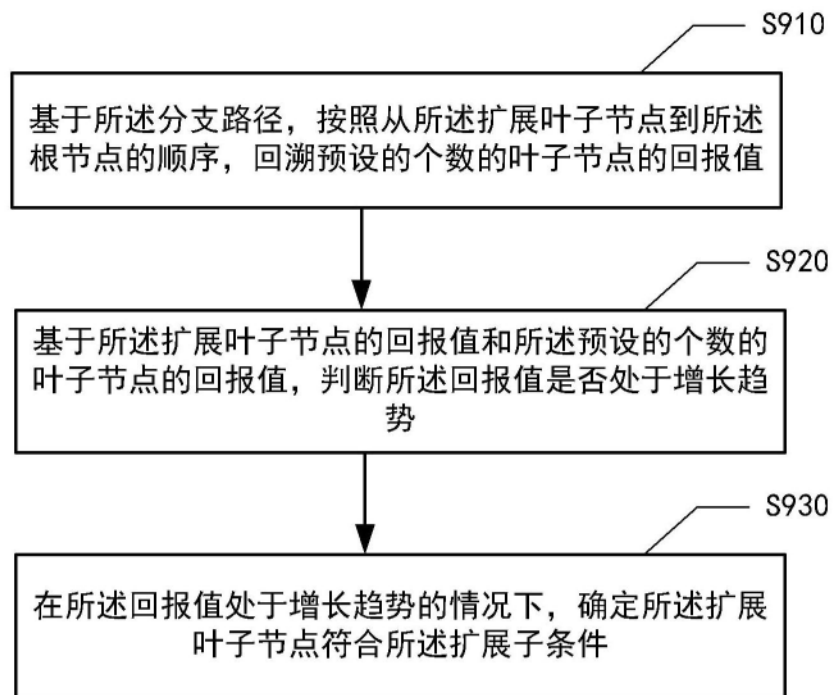


图9

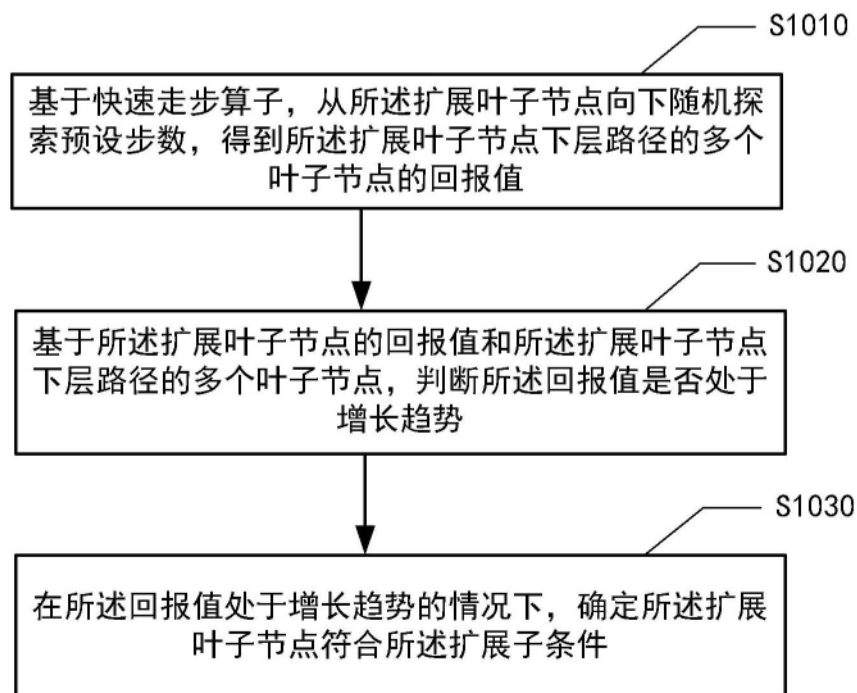


图10

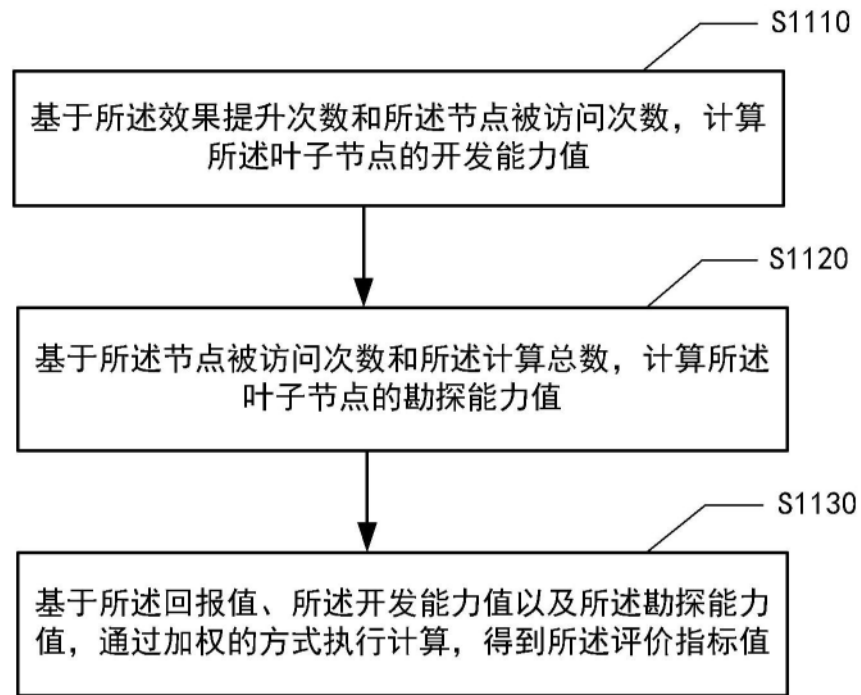


图11

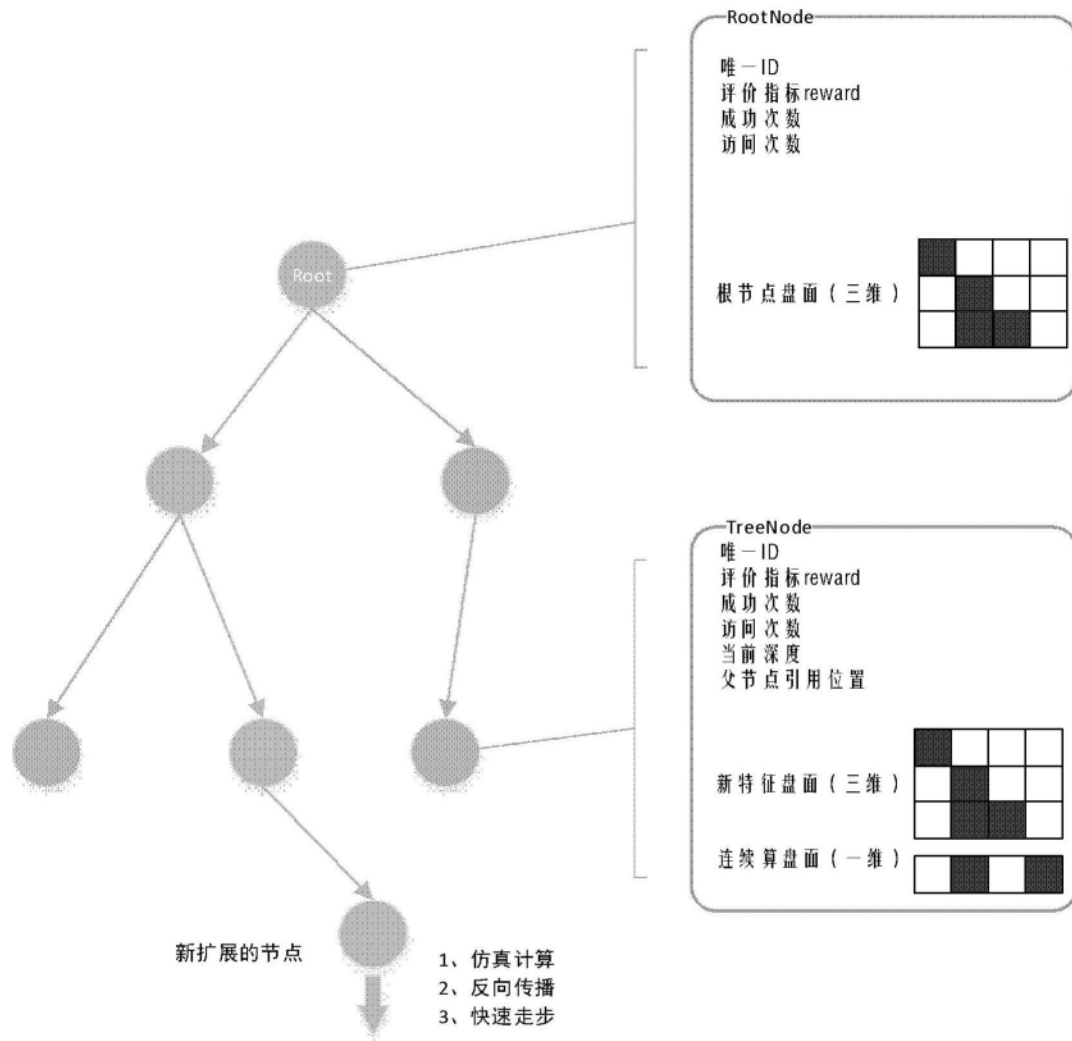


图12

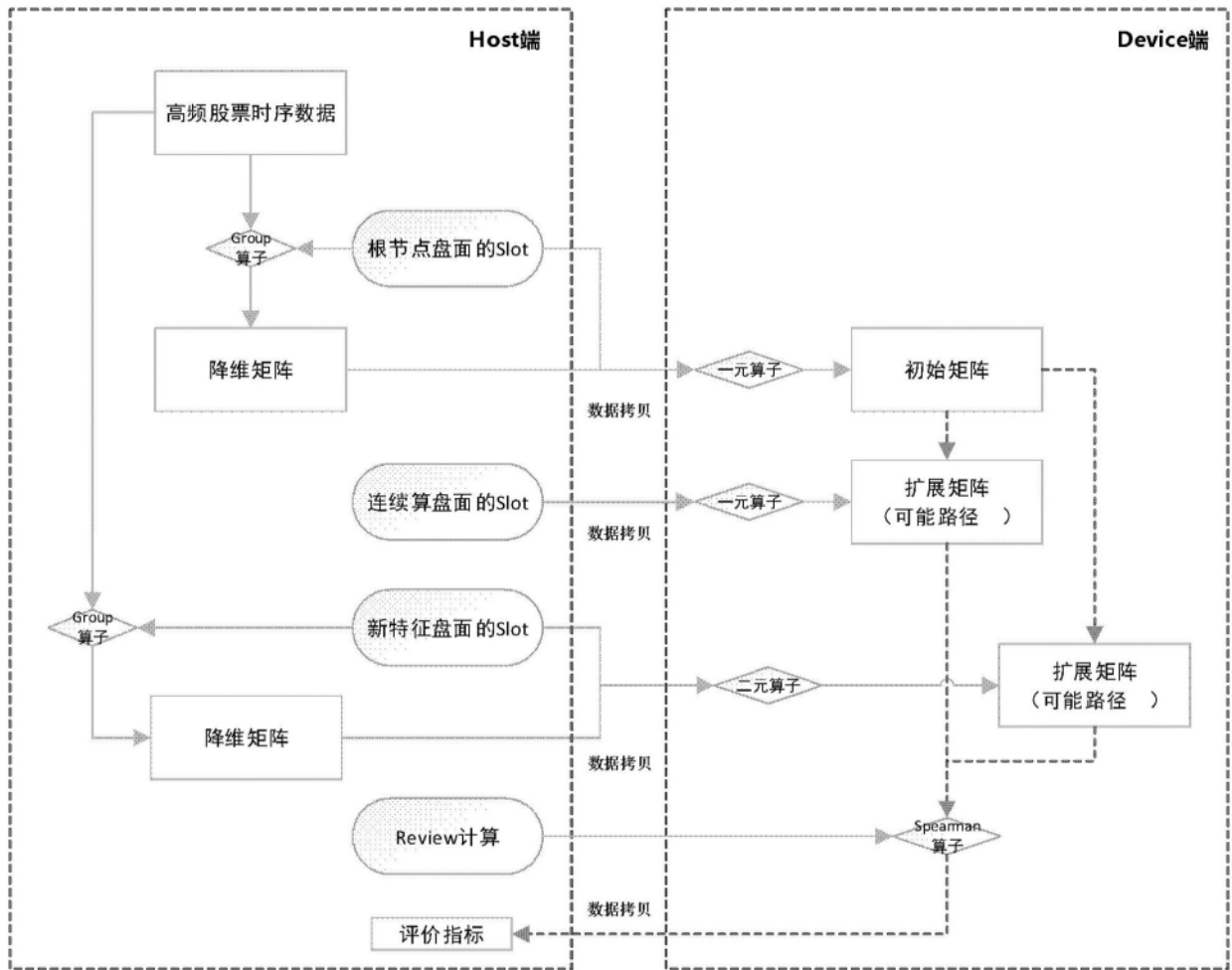


图13

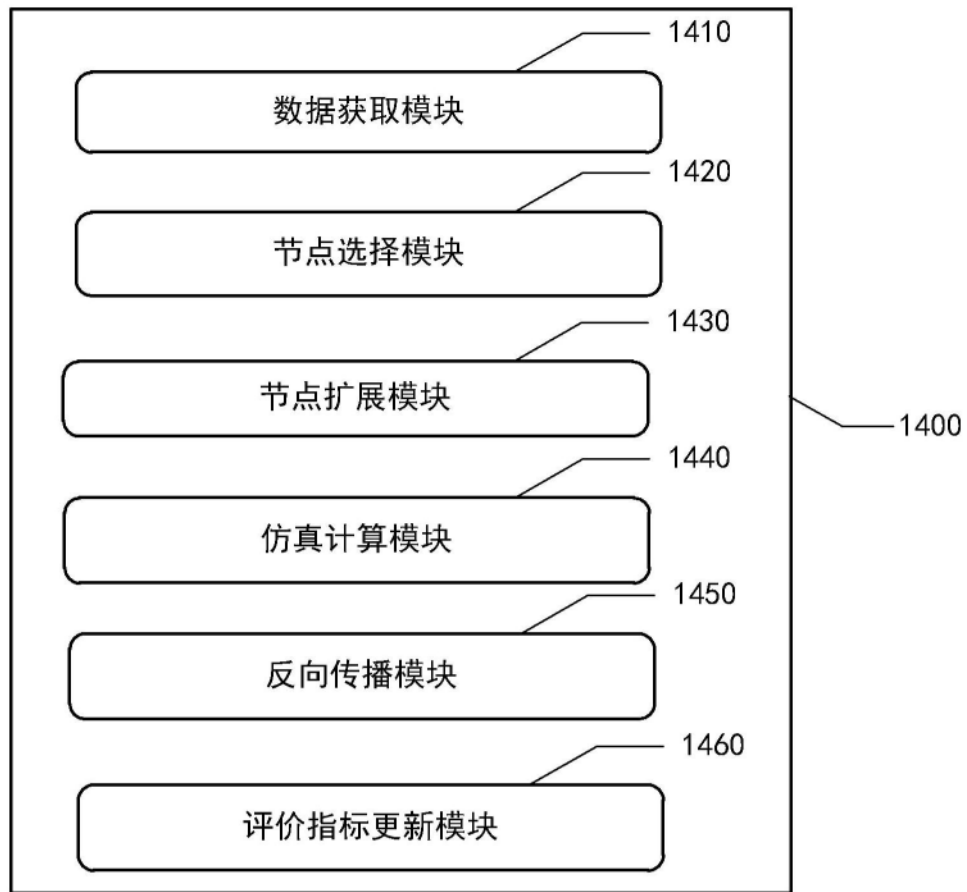


图14

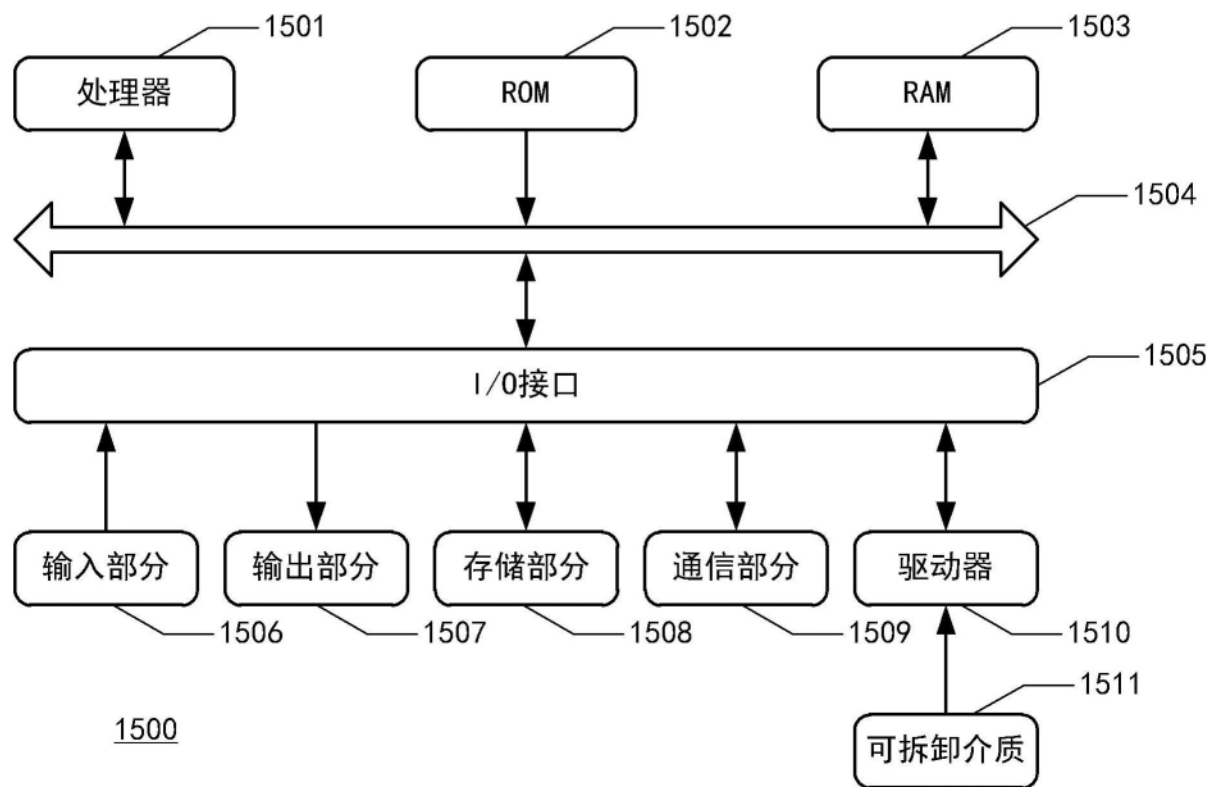


图15